

CHANGES-7: new features on your Midnight UI

You asked for: registration rules, browse-and-upload images, no second purchase (with "Go to course"), a footer, and deleting your own review. Nothing that already works was removed. Every file below is either **NEW** or an existing file with **small marked changes**, shown in full so you can paste it.

Read this first

Best way to upload images (my answer to your question). A database column cannot hold a picture, and a link to a picture breaks when the other website removes it. The best option for this project is: the admin picks the picture on his computer, the Spring Boot backend saves the file in an `uploads` folder, and the backend gives back a link to it. That link goes into the `courseImageUrl` field you already have. So the Course model, the course endpoints and the database stay the same. The alternatives (a cloud service like Cloudinary, or storing the picture as text in the database) need accounts, internet access or bigger columns, so they are not good for your team project.

Honest status - The Angular changes were built and tested in a real browser (68 checks passed, listed at the end) against a mock backend that behaves like your API. I did not run them against your real Spring Boot backend. - The Java changes are **not compiled or run**. My workspace cannot download Maven packages, so I have never been able to compile your backend. I wrote them carefully against your existing classes, but please run `mvn spring-boot:run` once and test upload and delete in Swagger before you rely on them. - **Hidden-test risk:** if your Examly tests type a link into the course image box (`#courseImageUrl`), or register users with simple passwords or phone numbers starting with 1 to 5, the new rules will make those tests fail. Check the SRS for the exact rules, and ask your SME before you merge.

Three rules to know 1. Register page rules apply only when someone registers. Old accounts still log in. 2. The server also refuses a second purchase of the same course (error 409), so it cannot be bypassed from outside the website. 3. Do **not** paste the whole `application.properties` over yours. Only add the lines in section B4. Your real DB password must stay out of GitHub: use the `DB_PASSWORD` environment variable.

Files

Backend (springapp)

- B1 `config/SecurityConfig.java` (3 small additions)
- B2 `service/OrderServiceImpl.java` (duplicate-purchase check)
- B3 `controller/ReviewController.java` (new delete-my-review endpoint)
- B4 `application.properties` (add 4 lines) and `.gitignore` (add 1 line)
- B5 NEW `controller/UploadController.java`
- B6 NEW `config/UploadConfig.java`

Frontend (angularapp/src)

- F1 `app/constant.ts` (2 lines)
- F2 NEW `app/services/image-upload.service.ts`
- F3 NEW `app/services/enrollment.service.ts`
- F4 NEW `app/services/review.service.ts`
- F5 NEW `app/components/footer/` (3 files)
- F6 `app/app.module.ts`, `app/app.component.html`, `app/app.component.css`
- F7 Registration rules: `registration.component.ts` and `.html`
- F8 Image browse: `add-course` and `edit-course` (`.ts` and `.html`)
- F9 Go to course: `course-card`, `customer-view-courses`, `course-details`
- F10 Cart and payment: `my-cart` (`.ts`, `.html`), `place-order.component.ts`
- F11 Delete my review: `add-review` (`.ts`, `.html`) (and `course-details`, already in F9)
- F12 CSS to append (6 files)

PART 1: BACKEND

B1. SecurityConfig.java

Changes: two new rules for uploads (read for everyone, upload for ADMIN) and one rule so a CUSTOMER can delete their own review. The "my" review rule must stay above the ADMIN delete rule, because the first matching rule wins. Search for `// NEW` free text: the added lines have comments.

`springapp/src/main/java/com/examly/springapp/config/SecurityConfig.java`

```
package com.examly.springapp.config;

import com.examly.springapp.repository.UserRepo;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.core.annotation.Order;
import org.springframework.http.HttpMethod;
import org.springframework.security.config.Customizer;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
import org.springframework.security.config.annotation.web.configurers.AbstractHttpConfigurer;
import org.springframework.security.config.http.SessionCreationPolicy;
import org.springframework.security.crypto.bcrypt.BCryptPasswordEncoder;
import org.springframework.security.crypto.password.PasswordEncoder;
import org.springframework.security.web.SecurityFilterChain;
import org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter;
import org.springframework.security.web.header.writers.ReferrerPolicyHeaderWriter;

/**
 * Who may call which endpoint. Two filter chains exist:
 * <ol>
```

```

* <li>Swagger/OpenAPI documentation paths (public, with a relaxed
* Content-Security-Policy so the UI can load)</li>
* <li>The API itself (JWT, roles, strict "default-src 'none'" policy)</li>
* </ol>
* In the API chain the ORDER of the requestMatchers matters: the first match
* wins, so specific rules come first.
*
* @author Suriya
*/
@Configuration
@EnableWebSecurity
public class SecurityConfig {
    /** Paths served by springdoc: the OpenAPI JSON/YAML and the Swagger UI. */
    private static final String[] SWAGGER_PATHS = {
        "/swagger-ui/**", "/swagger-ui.html", "/v3/api-docs", "/v3/api-docs/**", "/v3/api-docs.yaml"
    };

    /**
     * Swagger UI needs its own scripts, styles and inline styles; this is only used
     * for the documentation paths.
     */
    private static final String SWAGGER_CSP = "default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline'; "
        + "img-src 'self' data:; font-src 'self' data:; frame-ancestors 'none'";

    private final JwtUtil jwtUtil;
    private final UserRepo userRepo;
    private final RateLimiter rateLimiter;
    private final int globalPerMinute;
    private final int authPerMinute;

    public SecurityConfig(JwtUtil jwtUtil, UserRepo userRepo, RateLimiter rateLimiter,
        @Value("${ratelimit.global.per-minute:120}") int globalPerMinute,
        @Value("${ratelimit.auth.per-minute:10}") int authPerMinute) {
        this.jwtUtil = jwtUtil;
        this.userRepo = userRepo;
        this.rateLimiter = rateLimiter;
        this.globalPerMinute = globalPerMinute;
        this.authPerMinute = authPerMinute;
    }

    @Bean
    public PasswordEncoder passwordEncoder() {
        return new BCryptPasswordEncoder(12);
    }

    /**
     * Chain for the API documentation. It is matched first (Order 1) and only for
     * the Swagger paths, permits everyone and relaxes the Content-Security-Policy
     * for those paths only. The request-id and rate-limit filters still run, so the
     * docs cannot be used to flood the server.
     */
    @Bean
    @Order(1)
    public SecurityFilterChain swaggerFilterChain(HttpSecurity http) throws Exception {
        http
            .securityMatcher(SWAGGER_PATHS)
            .csrf(AbstractHttpConfigurer::disable)
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .headers(h -> h
                .contentSecurityPolicy(c -> c.policyDirectives(SWAGGER_CSP))
                .referrerPolicy(r -> r.policy(ReferrerPolicyHeaderWriter.ReferrerPolicy.NO_REFERRER)))
            .authorizeHttpRequests(auth -> auth.anyRequest().permitAll());
        http.addFilterBefore(new RateLimitFilter(rateLimiter, globalPerMinute, authPerMinute),
            UsernamePasswordAuthenticationFilter.class);
        http.addFilterBefore(new RequestIdFilter(), RateLimitFilter.class);
        return http.build();
    }

    @Bean
    @Order(2)
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        RestSecurityHandlers handlers = new RestSecurityHandlers();
        http
            // stateless API: no cookies authenticate API calls, only the Bearer token
            .csrf(AbstractHttpConfigurer::disable)
            .cors(Customizer.withDefaults())
            .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
            .headers(h -> h
                .contentSecurityPolicy(c -> c.policyDirectives("default-src 'none'; frame-ancestors 'none'"))
                .referrerPolicy(r -> r.policy(ReferrerPolicyHeaderWriter.ReferrerPolicy.NO_REFERRER)))
            .exceptionHandling(e -> e.authenticationEntryPoint(handlers).accessDeniedHandler(handlers))
            .authorizeHttpRequests(auth -> auth
                .requestMatchers(HttpMethod.OPTIONS, "**").permitAll()
                .requestMatchers("/api/register", "/api/login", "/api/refresh", "/api/logout", "/api/health")
                .permitAll()

                // API documentation (normally served by the swagger chain above; listed here
                // too so it is never blocked)
                .requestMatchers("/swagger-ui/**", "/swagger-ui.html", "/v3/api-docs", "/v3/api-docs/**",
                    "/v3/api-docs.yaml")
                .permitAll()

                // uploaded course images are public so the browser can show them in an <img> tag (read only)
                .requestMatchers(HttpMethod.GET, "/uploads/**").permitAll()

                // course image upload: admin only
                .requestMatchers(HttpMethod.POST, "/api/upload/**").hasAuthority("ADMIN")

                // admin management: only an existing admin can list, add or remove admins
                .requestMatchers("/api/admin/**").hasAuthority("ADMIN")

                // AI search: logged-in customers only (usage is limited per user)
                .requestMatchers("/api/course-ai/search").hasAuthority("CUSTOMER")

                // courses ("/api/course/search" is listed first for clarity; the general GET
                // rule below also covers it)
                .requestMatchers(HttpMethod.GET, "/api/course/search").hasAnyAuthority("ADMIN", "CUSTOMER")
                .requestMatchers(HttpMethod.GET, "/api/course", "/api/course/**")
                .hasAnyAuthority("ADMIN", "CUSTOMER")
                .requestMatchers("/api/course", "/api/course/**").hasAuthority("ADMIN")
            );
    }
}

```

```

        // customers (ownership is checked again inside the controller)
        .requestMatchers(HttpMethod.POST, "/api/customer").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.PUT, "/api/customer/**").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.GET, "/api/customer/user/**").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.GET, "/api/customer/**").hasAuthority("ADMIN")

        // orders
        .requestMatchers(HttpMethod.POST, "/api/order").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.GET, "/api/order").hasAuthority("ADMIN")
        .requestMatchers(HttpMethod.GET, "/api/order/customer/**").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.GET, "/api/order/**").hasAnyAuthority("ADMIN", "CUSTOMER")
        .requestMatchers(HttpMethod.DELETE, "/api/order/**").hasAuthority("CUSTOMER")

        // reviews
        .requestMatchers(HttpMethod.POST, "/api/review").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.GET, "/api/review/user/**").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.GET, "/api/review/summary", "/api/review/course/**")
        .hasAnyAuthority("ADMIN", "CUSTOMER")
        .requestMatchers(HttpMethod.GET, "/api/review", "/api/review/**")
        .hasAnyAuthority("ADMIN", "CUSTOMER")
        // a customer may delete only their own review (ownership is checked in the controller);
        // this rule must stay ABOVE the admin rule below because the first matching rule wins
        .requestMatchers(HttpMethod.DELETE, "/api/review/my/**").hasAuthority("CUSTOMER")
        .requestMatchers(HttpMethod.DELETE, "/api/review/**").hasAuthority("ADMIN")

        // dashboards: customers see their own numbers (checked again in the
        // controller), admins see the platform
        .requestMatchers(HttpMethod.GET, "/api/dashboard/admin").hasAuthority("ADMIN")
        .requestMatchers(HttpMethod.GET, "/api/dashboard/customer/**").hasAuthority("CUSTOMER")

        // carts
        .requestMatchers(HttpMethod.GET, "/api/cart").hasAuthority("ADMIN")
        .requestMatchers("/api/cart", "/api/cart/**").hasAuthority("CUSTOMER")

        .anyRequest().authenticated());

    // filter order: request id -> rate limit -> JWT
    http.addFilterBefore(new RateLimitFilter(rateLimiter, globalPerMinute, authPerMinute),
        UsernamePasswordAuthenticationFilter.class);
    http.addFilterBefore(new RequestIdFilter(), RateLimitFilter.class);
    http.addFilterAfter(new JwtRequestFilter(jwtUtil, userRepo), RateLimitFilter.class);
    return http.build();
}
}

```

B2. OrderServiceImpl.java

Changes: imports `DuplicateResourceException`, `HashSet`, `Set`, and the lines marked `// NEW` inside `addOrder`. A repeated course gives 409 with the message "You are already enrolled in ".

`springapp/src/main/java/com/examly/springapp/service/OrderServiceImpl.java`

```

package com.examly.springapp.service;

import com.examly.springapp.exception.DatabaseOperationException;
import com.examly.springapp.exception.DuplicateResourceException;
import com.examly.springapp.exception.InvalidRequestException;
import com.examly.springapp.exception.ResourceNotFoundException;
import com.examly.springapp.model.Course;
import com.examly.springapp.model.Customer;
import com.examly.springapp.model.Orders;
import com.examly.springapp.repository.CourseRepo;
import com.examly.springapp.repository.CustomerRepo;
import com.examly.springapp.repository.OrderRepo;
import java.util.ArrayList;
import java.util.HashSet;
import java.util.List;
import java.util.Set;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.stereotype.Service;

/**
 * Order rules: an order needs a customer and at least one existing course, and its price is always
 * computed on the server. Repository failures are wrapped into DatabaseOperationException.
 *
 * @author Tanvi
 */
@Service
public class OrderServiceImpl implements OrderService {
    private static final Logger log = LoggerFactory.getLogger(OrderServiceImpl.class);

    private final OrderRepo orderRepo;
    private final CustomerRepo customerRepo;
    private final CourseRepo courseRepo;

    public OrderServiceImpl(OrderRepo orderRepo, CustomerRepo customerRepo, CourseRepo courseRepo) {
        this.orderRepo = orderRepo;
        this.customerRepo = customerRepo;
        this.courseRepo = courseRepo;
    }

    @Override
    public Orders addOrder(Orders order) {
        log.trace("addOrder entered");
        if (order.getCustomer() == null || order.getCustomer().getCustomerId() == null) {
            throw new InvalidRequestException("A customer id is required");
        }
        if (order.getCourses() == null || order.getCourses().isEmpty()) {
            throw new InvalidRequestException("An order needs at least one course");
        }
        Long customerId = order.getCustomer().getCustomerId();
        Customer customer = DatabaseOperationException.guard("loading a customer", () -> customerRepo.findById(customerId))
    }
}

```

```

        .orElseThrow(() -> new ResourceNotFoundException("Customer not found with id " + customerId));
// NEW: a course can be bought only once. Collect what this customer has already bought.
Set<Long> alreadyBought = new HashSet<>();
DatabaseOperationException.guard("loading orders", () -> orderRepo.findByCustomer_CustomerId(customerId))
    .forEach(o -> o.getCourses().forEach(c -> alreadyBought.add(c.getCourseId())));
Set<Long> inThisOrder = new HashSet<>();
List<Course> courses = new ArrayList<>();
double total = 0;
for (Course c : order.getCourses()) {
    if (c == null || c.getCourseId() == null) {
        throw new InvalidRequestException("Each course must have a courseId");
    }
    Long courseId = c.getCourseId();
    Course found = DatabaseOperationException.guard("loading a course", () -> courseRepo.findById(courseId))
        .orElseThrow(() -> new ResourceNotFoundException("Course not found with id " + courseId));
// NEW: refuse a course the customer already owns, and the same course twice in one order
if (alreadyBought.contains(courseId) || !inThisOrder.add(courseId)) {
    throw new DuplicateResourceException("You are already enrolled in " + found.getCourseType());
}
    courses.add(found);
    total += found.getCoursePrice() == null ? 0 : found.getCoursePrice();
}
log.debug("addOrder: customerId={}, courses={}, computed total={}", customerId, courses.size(), total);
order.setOrderId(null);
order.setCustomer(customer);
order.setCourses(courses);
order.setOrderPrice(total); // price is always computed on the server
Orders saved = DatabaseOperationException.guard("saving an order", () -> orderRepo.save(order));
log.info("Order {} created for customer {}", saved.getOrderId(), customerId);
return saved;
}

@Override
public List<Orders> getAllOrders() {
    log.trace("getAllOrders entered");
    List<Orders> orders = DatabaseOperationException.guard("loading orders", () -> orderRepo.findAll());
    log.debug("getAllOrders returned {} order(s)", orders.size());
    return orders;
}

@Override
public Orders getOrderByById(Long orderId) {
    log.trace("getOrderByById entered for orderId={}", orderId);
    return DatabaseOperationException.guard("loading an order", () -> orderRepo.findById(orderId))
        .orElseThrow(() -> new ResourceNotFoundException("Order not found with id " + orderId));
}

@Override
public List<Orders> getOrdersByCustomerId(Long customerId) {
    log.trace("getOrdersByCustomerId entered for customerId={}", customerId);
    List<Orders> orders = DatabaseOperationException.guard("loading orders", () -> orderRepo.findByCustomer_CustomerId(customerId));
    log.debug("getOrdersByCustomerId: {} order(s) for customer {}", orders.size(), customerId);
    if (orders.isEmpty()) {
        throw new ResourceNotFoundException("No orders found for customer id " + customerId);
    }
    return orders;
}

@Override
public Orders deleteOrder(Long orderId) {
    log.trace("deleteOrder entered for orderId={}", orderId);
    Orders order = getOrderByById(orderId);
    DatabaseOperationException.guardVoid("deleting an order", () -> orderRepo.delete(order));
    log.info("Order {} deleted", orderId);
    return order;
}
}

```

B3. ReviewController.java

Changes: one new import and the new method `deleteMyReview` at the bottom (DELETE `/api/review/my/{reviewId}`, CUSTOMER only, own review only, otherwise 403). The existing admin delete is untouched.

```
springapp/src/main/java/com/examly/springapp/controller/ReviewController.java
```

```

package com.examly.springapp.controller;

import com.examly.springapp.dto.CourseRatingDTO;
import com.examly.springapp.model.Review;
import com.examly.springapp.service.AccessService;
import com.examly.springapp.service.ReviewService;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import jakarta.validation.Valid;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;
import java.util.List;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.security.access.AccessDeniedException;
import org.springframework.validation.annotation.Validated;
import org.springframework.web.bind.annotation.*;

/**
 * Review endpoints.
 *
 * <p><b>Where 204 No Content is used.</b> Only the NEW endpoint GET /api/review/course/{courseType} answers
 * 204 when a course has no reviews yet, because "nothing to show" is a normal situation there.
 * The endpoints that come from the SRS API table keep their documented behaviour (200 with data, 404 when the

```

```

* record or list is missing, e.g. GET /api/review/user/{userId}), so the Angular app and the SRS stay valid.
*
* @author Sumit
*/
@RestController
@RequestMapping("/api/review")
@Validated
@Tag(name = "Reviews", description = "Course reviews and rating summary")
public class ReviewController {
    private static final Logger log = LoggerFactory.getLogger(ReviewController.class);

    private final ReviewService reviewService;
    private final AccessService access;

    @Autowired
    public ReviewController(ReviewService reviewService, AccessService access) {
        this.reviewService = reviewService;
        this.access = access;
    }

    @Operation(summary = "Add a review", description = "CUSTOMER only. The subject must be the type of a course the customer has bought.")
    @ApiResponse({
        @ApiResponse(responseCode = "201", description = "Review created"),
        @ApiResponse(responseCode = "400", description = "Validation failed or course not bought"),
        @ApiResponse(responseCode = "401", description = "Missing or invalid token"),
        @ApiResponse(responseCode = "403", description = "Not a customer"),
        @ApiResponse(responseCode = "404", description = "Customer profile not found")
    })
    @PostMapping
    public ResponseEntity<Review> addReview(@Valid @RequestBody Review review) {
        log.trace("addReview called");
        review.setCustomer(access.currentCustomer());
        Review saved = reviewService.addReview(review);
        return ResponseEntity.status(HttpStatus.CREATED).body(saved);
    }

    @Operation(summary = "List all reviews", description = "ADMIN and CUSTOMER.")
    @ApiResponse({
        @ApiResponse(responseCode = "200", description = "List of reviews (may be empty)"),
        @ApiResponse(responseCode = "401", description = "Missing or invalid token")
    })
    @GetMapping
    public ResponseEntity<List<Review>> getAllReviews() {
        log.trace("getAllReviews called");
        List<Review> reviews = reviewService.getAllReviews();
        log.debug("getAllReviews: returning {} review(s)", reviews.size());
        return ResponseEntity.status(HttpStatus.OK).body(reviews);
    }

    /** Declared before("/{reviewId}"; the literal path "summary" always wins over the variable path. */
    @Operation(summary = "Rating summary per course",
        description = "ADMIN and CUSTOMER. One row per course: {courseType, averageRating (1 decimal), reviewCount}."
    )
    @ApiResponse({
        @ApiResponse(responseCode = "200", description = "Summary list (may be empty)"),
        @ApiResponse(responseCode = "401", description = "Missing or invalid token")
    })
    @GetMapping("/{summary}")
    public ResponseEntity<List<CourseRatingDTO>> getRatingSummary() {
        log.trace("getRatingSummary called");
        List<CourseRatingDTO> summary = reviewService.getRatingSummary();
        log.debug("getRatingSummary: returning {} row(s)", summary.size());
        return ResponseEntity.status(HttpStatus.OK).body(summary);
    }

    @Operation(summary = "Reviews of one course",
        description = "ADMIN and CUSTOMER. Reviews whose subject equals the course type (ignoring case). 204 when there are none."
    )
    @ApiResponse({
        @ApiResponse(responseCode = "200", description = "List of reviews"),
        @ApiResponse(responseCode = "204", description = "The course has no reviews"),
        @ApiResponse(responseCode = "400", description = "Course type is blank"),
        @ApiResponse(responseCode = "401", description = "Missing or invalid token")
    })
    @GetMapping("/{course/{courseType}")
    public ResponseEntity<List<Review>> getReviewsByCourseType(
        @PathVariable @NotBlank(message = "must not be blank") String courseType) {
        log.trace("getReviewsByCourseType called for '{}', courseType");
        List<Review> reviews = reviewService.getReviewsByCourseType(courseType);
        if (reviews.isEmpty()) {
            log.debug("getReviewsByCourseType: no reviews for '{}', answering 204", courseType);
            return ResponseEntity.status(HttpStatus.NO_CONTENT).build();
        }
        return ResponseEntity.status(HttpStatus.OK).body(reviews);
    }

    @Operation(summary = "Get a review by id", description = "ADMIN and CUSTOMER.")
    @ApiResponse({
        @ApiResponse(responseCode = "200", description = "Review found"),
        @ApiResponse(responseCode = "400", description = "Invalid id"),
        @ApiResponse(responseCode = "401", description = "Missing or invalid token"),
        @ApiResponse(responseCode = "404", description = "Review not found")
    })
    @GetMapping("/{reviewId}")
    public ResponseEntity<Review> getReviewById(@PathVariable @Min(value = 1, message = "must be a positive number") Long reviewId) {
        log.trace("getReviewById called for reviewId={}", reviewId);
        return ResponseEntity.status(HttpStatus.OK).body(reviewService.getReviewById(reviewId));
    }

    @Operation(summary = "Reviews written by a user", description = "CUSTOMER only, own data.")
    @ApiResponse({
        @ApiResponse(responseCode = "200", description = "List of reviews"),
        @ApiResponse(responseCode = "400", description = "Invalid id"),
        @ApiResponse(responseCode = "401", description = "Missing or invalid token"),
        @ApiResponse(responseCode = "403", description = "Not your data"),
        @ApiResponse(responseCode = "404", description = "No reviews found for this user")
    })
    @GetMapping("/{user/{userId}")
    public ResponseEntity<List<Review>> getReviewsByUserId(@PathVariable @Min(value = 1, message = "must be a positive number") Long userId) {
        log.trace("getReviewsByUserId called for userId={}", userId);
        access.requireUserAccess(userId);
        return ResponseEntity.status(HttpStatus.OK).body(reviewService.getReviewsByUserId(userId));
    }
}

```

```

    }

    @Operation(summary = "Delete a review", description = "ADMIN only.")
    @ApiResponse({
        @ApiResponse(responseCode = "200", description = "Review deleted, the deleted review is returned"),
        @ApiResponse(responseCode = "400", description = "Invalid id"),
        @ApiResponse(responseCode = "401", description = "Missing or invalid token"),
        @ApiResponse(responseCode = "403", description = "Not an admin"),
        @ApiResponse(responseCode = "404", description = "Review not found")
    })
    @DeleteMapping("/{reviewId}")
    public ResponseEntity<Review> deleteReview(@PathVariable @Min(value = 1, message = "must be a positive number") Long reviewId) {
        log.trace("deleteReview called for reviewId={}", reviewId);
        return ResponseEntity.status(HttpStatus.OK).body(reviewService.deleteReview(reviewId));
    }

    @Operation(summary = "Delete my own review", description = "CUSTOMER only. A customer can remove only a review they wrote.")
    @ApiResponse({
        @ApiResponse(responseCode = "200", description = "Review deleted, the deleted review is returned"),
        @ApiResponse(responseCode = "400", description = "Invalid id"),
        @ApiResponse(responseCode = "401", description = "Missing or invalid token"),
        @ApiResponse(responseCode = "403", description = "Not a customer, or the review belongs to someone else"),
        @ApiResponse(responseCode = "404", description = "Review not found")
    })
    @DeleteMapping("/my/{reviewId}")
    public ResponseEntity<Review> deleteMyReview(@PathVariable @Min(value = 1, message = "must be a positive number") Long reviewId) {
        log.trace("deleteMyReview called for reviewId={}", reviewId);
        Review review = reviewService.getReviewById(reviewId);
        Long ownerId = review.getCustomerId() == null ? null : review.getCustomerId().getCustomerId();
        if (ownerId == null || !ownerId.equals(access.currentCustomer().getCustomerId())) {
            throw new AccessDeniedException("You can only delete your own review");
        }
        return ResponseEntity.status(HttpStatus.OK).body(reviewService.deleteReview(reviewId));
    }
}

```

B4. application.properties and .gitignore

Add these lines at the bottom of `springapp/src/main/resources/application.properties` (do not replace the file):

```

# ---- Image upload (admin course images) ----
app.upload.dir=${UPLOAD_DIR:uploads}
spring.servlet.multipart.max-file-size=20MB
spring.servlet.multipart.max-request-size=21MB

```

The controller allows images up to 5 MB. The 20 MB multipart limit is only a ceiling, so that a too-big file gets a clear "too large" message instead of a server error.

Add this line to `.gitignore` in the repo root, so uploaded pictures are not committed:

```

springapp/uploads/
uploads/

```

Where do the files go? The `uploads` folder is created next to where you start the backend (your log shows it starts in `/home/coder/project/workspace`, so it becomes `/home/coder/project/workspace/uploads`). Set `UPLOAD_DIR` to use another folder.

B5. NEW UploadController.java

`POST /api/upload/image` (ADMIN). It checks the first bytes of the file to be sure it is a PNG, JPG, GIF or WEBP, limits it to 5 MB, saves it under a random name and returns `{ "url": "http://localhost:8080/uploads/<name>" }`.

`springapp/src/main/java/com/examly/springapp/controller/UploadController.java`

```

package com.examly.springapp.controller;

import com.examly.springapp.exception.InvalidRequestException;
import io.swagger.v3.oas.annotations.Operation;
import io.swagger.v3.oas.annotations.responses.ApiResponse;
import io.swagger.v3.oas.annotations.responses.ApiResponses;
import io.swagger.v3.oas.annotations.tags.Tag;
import java.io.IOException;
import java.io.InputStream;
import java.io.UncheckedIOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import java.nio.file.StandardCopyOption;
import java.util.Map;
import java.util.UUID;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.http.MediaType;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;
import org.springframework.web.multipart.MultipartFile;
import org.springframework.web.servlet.support.ServletUriComponentsBuilder;

/**
 * Course image upload. The admin picks a picture in the browser, this endpoint saves it in the uploads folder and
 * answers with its public URL. That URL is then stored in the existing Course.courseImageUrl field, so the Course
 * model and the course endpoints do not change.
 *
 * <p>Safety: the file type is detected from the first bytes of the file (the name and Content-Type sent by the
 * browser are never trusted), the size is limited, and the stored name is a random UUID chosen by the server.
 *

```

```

* @author Team Lead
*/
@RestController
@RequestMapping("/api/upload")
@Tag(name = "Uploads", description = "Course image upload")
public class UploadController {
    private static final Logger log = LoggerFactory.getLogger(UploadController.class);
    private static final long MAX_BYTES = 5L * 1024 * 1024;    // 5 MB

    private final Path uploadDir;

    public UploadController(@Value("${app.upload.dir:uploads}") String dir) throws IOException {
        this.uploadDir = Paths.get(dir).toAbsolutePath().normalize();
        Files.createDirectories(this.uploadDir);
    }

    @Operation(summary = "Upload a course image", description = "ADMIN only. PNG, JPG, GIF or WEBP, up to 5 MB. Returns the public URL.")
    @ApiResponse(responseCode = "200", description = "Image saved, {\\"url\\": \\"...\\"} returned"),
    @ApiResponse(responseCode = "400", description = "Empty file, wrong type or too large"),
    @ApiResponse(responseCode = "401", description = "Missing or invalid token"),
    @ApiResponse(responseCode = "403", description = "Not an admin")
    @PostMapping(value = "/image", consumes = MediaType.MULTIPART_FORM_DATA_VALUE)
    public ResponseEntity<Map<String, String>> uploadImage(@RequestParam("file") MultipartFile file) {
        if (file == null || file.isEmpty()) {
            throw new InvalidRequestException("Please choose an image file");
        }
        if (file.getSize() > MAX_BYTES) {
            throw new InvalidRequestException("The image is too large. The limit is 5 MB");
        }
        String extension = detectExtension(file);
        if (extension == null) {
            throw new InvalidRequestException("Only PNG, JPG, GIF or WEBP images are allowed");
        }
        String storedName = UUID.randomUUID() + "." + extension;
        Path target = uploadDir.resolve(storedName).normalize();
        if (!target.startsWith(uploadDir)) {
            throw new InvalidRequestException("Invalid file name");
        }
        try (InputStream in = file.getInputStream()) {
            Files.copy(in, target, StandardCopyOption.REPLACE_EXISTING);
        } catch (IOException e) {
            log.error("Could not save an uploaded image", e);
            throw new UncheckedIOException("Could not save the image", e);
        }
        String url = ServletUriComponentsBuilder.fromCurrentContextPath().path("/uploads/").path(storedName).toUriString();
        log.info("Course image saved as {}", storedName);
        return ResponseEntity.ok(Map.of("url", url));
    }

    /** Looks at the first bytes of the file and returns png, jpg, gif or webp; null when it is none of these. */
    private String detectExtension(MultipartFile file) {
        byte[] h = new byte[12];
        int read;
        try (InputStream in = file.getInputStream()) {
            read = in.readNBytes(h, 0, h.length);
        } catch (IOException e) {
            throw new UncheckedIOException("Could not read the image", e);
        }
        if (read < 12) {
            return null;
        }
        if ((h[0] & 0xFF) == 0x89 && h[1] == 'P' && h[2] == 'N' && h[3] == 'G') {
            return "png";
        }
        if ((h[0] & 0xFF) == 0xFF && (h[1] & 0xFF) == 0xD8 && (h[2] & 0xFF) == 0xFF) {
            return "jpg";
        }
        if (h[0] == 'G' && h[1] == 'I' && h[2] == 'F' && h[3] == '8') {
            return "gif";
        }
        if (h[0] == 'R' && h[1] == 'I' && h[2] == 'F' && h[3] == 'F' && h[8] == 'W' && h[9] == 'E' && h[10] == 'B' && h[11] == 'P') {
            return "webp";
        }
        return null;
    }
}

```

B6. NEW UploadConfig.java

Makes the `uploads` folder readable at `/uploads/**`, so the browser can show the pictures.

`springapp/src/main/java/com/examly/springapp/config/UploadConfig.java`

```

package com.examly.springapp.config;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Path;
import java.nio.file.Paths;
import org.springframework.beans.factory.annotation.Value;
import org.springframework.context.annotation.Configuration;
import org.springframework.http.CacheControl;
import org.springframework.web.servlet.config.annotation.ResourceHandlerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;
import java.util.concurrent.TimeUnit;

/**
 * Serves the uploaded course images at /uploads/** (read only, public, so the browser can show them in an img tag).
 * The folder is set by app.upload.dir (default: "uploads" next to where the app is started).
 *
 * @author Team Lead
 */
@Configuration

```

```

public class UploadConfig implements WebMvcConfigurer {
    private final Path uploadDir;

    public UploadConfig(@Value("${app.upload.dir:uploads}") String dir) throws IOException {
        this.uploadDir = Paths.get(dir).toAbsolutePath().normalize();
        Files.createDirectories(this.uploadDir);
    }

    @Override
    public void addResourceHandlers(ResourceHandlerRegistry registry) {
        registry.addResourceHandler("/uploads/**")
            .addResourceLocations(uploadDir.toUri().toString())
            .setCacheControl(CacheControl.maxAge(7, TimeUnit.DAYS).cachePublic());
    }
}

```

PART 2: FRONTEND

F1. constant.ts

Two lines added inside API: `UPLOAD_IMAGE` and `REVIEW_MY`.

`angularapp/src/app/constant.ts`

```

/**
 * Every API URL, storage key and role name of the app lives here, so nothing is hard-coded in components or services.
 * The host part (http://localhost:8080) comes from environments/environment.ts.
 */
 * @author Team Lead
 */
import { environment } from '../environments/environment';

export const API_URL: string = environment.apiUrl;

export const API = {
    // authentication (Suriya)
    REGISTER: '/api/register',
    LOGIN: '/api/login',
    REFRESH: '/api/refresh',
    LOGOUT: '/api/logout',
    ADMIN_USERS: '/api/admin/users',

    // courses (Sivamuthu, Amogh)
    COURSE: '/api/course',
    COURSE_SEARCH: '/api/course/search',
    COURSE_AI_SEARCH: '/api/course-ai/search',
    UPLOAD_IMAGE: '/api/upload/image',

    // customer profile and dashboard (Meruva Lokes)
    CUSTOMER: '/api/customer',
    CUSTOMER_BY_USER: '/api/customer/user',
    DASHBOARD_CUSTOMER: '/api/dashboard/customer',
    DASHBOARD_ADMIN: '/api/dashboard/admin',

    // cart (Shoryan)
    CART: '/api/cart',
    CART_BY_CUSTOMER: '/api/cart/customer',
    CART_BY_USER: '/api/cart/user',

    // orders (Tanvi)
    ORDER: '/api/order',
    ORDER_BY_CUSTOMER: '/api/order/customer',

    // reviews (Sumit)
    REVIEW: '/api/review',
    REVIEW_BY_USER: '/api/review/user',
    REVIEW_BY_COURSE: '/api/review/course',
    REVIEW_SUMMARY: '/api/review/summary',
    REVIEW_MY: '/api/review/my'
};

/** localStorage keys (the SRS keeps the JWT in localStorage). */
export const STORAGE = {
    TOKEN: 'token',
    ROLE: 'role',
    USER_ID: 'userId',
    CUSTOMER_ID: 'customerId',
    CART_ID: 'cartId',
    SESSION: 'session'
};

export const ROLES = {
    ADMIN: 'ADMIN',
    CUSTOMER: 'CUSTOMER'
};

/** Paths used by the AuthInterceptor to skip the Bearer header. */
export const PUBLIC_API_PATTERN = /\api\/(login|register|refresh|logout)$/;

export const PAGE_SIZE = 9;

```

F2 to F4. New services

Three new files in `angularapp/src/app/services/`. They do not change any existing service.

`angularapp/src/app/services/image-upload.service.ts`

```

import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable } from 'rxjs';
import { map } from 'rxjs/operators';
import { API, API_URL } from '../constant';

/**
 * Course image upload. The admin browses for a picture, it is sent to the server, and the server answers with the
 * public URL. That URL is saved in the course's existing courseImageUrl field, so the Course model does not change.
 */
@Injectable({
  providedIn: 'root'
})
export class ImageUploadService {
  public apiUrl = API_URL;

  /** Same limits as the server (the server checks again, this only gives a faster message). */
  readonly maxBytes = 5 * 1024 * 1024;
  readonly allowedTypes = ['image/png', 'image/jpeg', 'image/gif', 'image/webp'];

  constructor(private http: HttpClient) {}

  /** Returns an error message, or null when the file can be uploaded. */
  validate(file: File): string | null {
    if (!this.allowedTypes.includes(file.type)) { return 'Choose a PNG, JPG, GIF or WEBP image'; }
    if (file.size > this.maxBytes) { return 'The image is too large. The limit is 5 MB'; }
    return null;
  }

  /** Sends the file and returns its public URL. The token is added by the AuthInterceptor. */
  upload(file: File): Observable<string> {
    const body = new FormData();
    body.append('file', file);
    return this.http.post<{ url: string }>(`${this.apiUrl}${API.UPLOAD_IMAGE}`, body).pipe(map(res => res.url));
  }
}

```

angularapp/src/app/services/enrollment.service.ts

```

import { Injectable } from '@angular/core';
import { Observable, of } from 'rxjs';
import { catchError, map } from 'rxjs/operators';
import { OrderService } from './order.service';

/**
 * Which courses has the logged-in customer already bought? Used to show "Go to course" instead of
 * "Add to Cart" so the same course cannot be bought twice (the server refuses it too).
 */
@Injectable({
  providedIn: 'root'
})
export class EnrollmentService {
  constructor(private orders: OrderService) {}

  /** Ids of the bought courses. An empty set when there are no orders (the API answers 404 then) or on any error. */
  enrolledCourseIds(): Observable<Set<number>> {
    return this.orders.viewOrderByCustomerId().pipe(
      map((orders: any[]) => {
        const ids = new Set<number>();
        (orders || []).forEach(o => (o.courses || []).forEach((c: any) => { if (c.courseId != null) { ids.add(c.courseId); } }));
        return ids;
      }),
      catchError(() => of(new Set<number>()))
    );
  }
}

```

angularapp/src/app/services/review.service.ts

```

import { HttpClient } from '@angular/common/http';
import { Injectable } from '@angular/core';
import { Observable, of } from 'rxjs';
import { catchError } from 'rxjs/operators';
import { API, API_URL, STORAGE } from '../constant';
import { Review } from '../models/review.model';

/**
 * A customer's own reviews: list them and delete one. (Adding and reading reviews stays in CartService.)
 */
@Injectable({
  providedIn: 'root'
})
export class ReviewService {
  public apiUrl = API_URL;

  constructor(private http: HttpClient) {}

  /** The logged-in customer's reviews. The server answers 404 when there are none, which is shown as an empty list. */
  getMyReviews(): Observable<Review[]> {
    const userId = localStorage.getItem(STORAGE.USER_ID);
    return this.http.get<Review[]>(`${this.apiUrl}${API.REVIEW_BY_USER}/${userId}`).pipe(catchError(() => of([])));
  }

  /** Deletes one of my own reviews (the server refuses somebody else's review with 403). */
  deleteMyReview(reviewId: number): Observable<Review> {
    return this.http.delete<Review>(`${this.apiUrl}${API.REVIEW_MY}/${reviewId}`);
  }
}

```

F5. Footer (3 new files)

Create the folder `angularapp/src/app/components/footer/` with these files. The footer shows quick links that change with the role (visitor, customer, admin).

`angularapp/src/app/components/footer/footer.component.ts`

```
import { Component, OnDestroy, OnInit } from '@angular/core';
import { Subscription } from 'rxjs';
import { AuthService } from '../../../services/auth.service';

/** Site footer shown under every page. The quick links change with the role of the logged-in user. */
@Component({
  selector: 'app-footer',
  templateUrl: './footer.component.html',
  styleUrls: ['./footer.component.css']
})
export class FooterComponent implements OnInit, OnDestroy {
  role: string | null = null;
  year = new Date().getFullYear();
  private sub?: Subscription;

  constructor(private auth: AuthService) {}

  ngOnInit(): void {
    this.sub = this.auth.role$.subscribe(r => (this.role = r));
  }

  ngOnDestroy(): void {
    this.sub?.unsubscribe();
  }
}
```

`angularapp/src/app/components/footer/footer.component.html`

```
<footer class="foot" role="contentinfo">
  <div class="foot-inner">
    <div class="about">
      <a class="brand" [routerLink]="role ? '/dashboard' : '/'"><mat-icon>school</mat-icon><span>Online E-Learning Platform</span></a>
      <p>Learn at your own pace with courses, AI-powered search and real learner reviews.</p>
    </div>

    <nav class="col" aria-label="Quick links">
      <h2>Quick links</h2>
      <ng-container *ngIf="!role">
        <a routerLink="/">Home</a>
        <a routerLink="/registration">Register</a>
        <a routerLink="/login">Login</a>
      </ng-container>
      <ng-container *ngIf="role === 'CUSTOMER'">
        <a routerLink="/dashboard">Home</a>
        <a routerLink="/customer-view-courses">Available Courses</a>
        <a routerLink="/my-orders">My Enrollments</a>
        <a routerLink="/my-cart">My Cart</a>
      </ng-container>
      <ng-container *ngIf="role === 'ADMIN'">
        <a routerLink="/dashboard">Home</a>
        <a routerLink="/view-courses">View Courses</a>
        <a routerLink="/add-course">Add Course</a>
        <a routerLink="/view-orders">View Enrollments</a>
      </ng-container>
    </nav>

    <div class="col">
      <h2>Why learn here</h2>
      <span><mat-icon>menu_book</mat-icon>Wide range of courses</span>
      <span><mat-icon>auto_awesome</mat-icon>AI course search</span>
      <span><mat-icon>star</mat-icon>Real learner reviews</span>
    </div>
  </div>
  <div class="legal">&copy; {{ year }} Online E-Learning Platform. All rights reserved.</div>
</footer>
```

`angularapp/src/app/components/footer/footer.component.css`

```
/* Footer (Midnight): sits a little darker than the page, with a gold hairline on top */
.foot { margin-top: 48px; background: #0a101c; border-top: 1px solid rgba(245, 158, 11, .28); color: var(--muted); }
.foot-inner { display: grid; grid-template-columns: 1.6fr 1fr 1fr; gap: 40px; max-width: 1200px; margin: 0 auto; padding: 40px 20px 28px; }
.brand { display: inline-flex; align-items: center; gap: 10px; color: var(--ink); text-decoration: none; font: 800 17px var(--display); }
.brand mat-icon { display: grid; place-items: center; width: 34px; height: 34px; font-size: 20px; border-radius: 9px; background: var(--gold); color: #1a1205; }
.about p { margin: 14px 0 0; max-width: 40ch; line-height: 1.65; font-size: 14.5px; }
.col { display: flex; flex-direction: column; gap: 10px; }
.col h2 { margin-bottom: 4px; font: 700 13px var(--display); letter-spacing: .06em; color: var(--gold-text); }
.col a { color: var(--ink-2); text-decoration: none; font-weight: 500; font-size: 14.5px; width: fit-content; }
.col a:hover { color: var(--gold-text); text-decoration: underline; text-underline-offset: 3px; }
.col span { display: flex; align-items: center; gap: 8px; font-size: 14.5px; }
.col span mat-icon { width: 18px; height: 18px; font-size: 18px; color: var(--gold); }
.legal { max-width: 1200px; margin: 0 auto; padding: 18px 20px 26px; border-top: 1px solid var(--line); font-size: 13px; }
@media (max-width: 760px) { .foot-inner { grid-template-columns: 1fr; gap: 28px; padding-top: 32px; } }
```

F6. Show the footer on every page

`app.module.ts` declares `FooterComponent`. `app.component.html` adds `<app-footer>`. `app.component.css` keeps the footer at the bottom of short pages.

`angularapp/src/app/app.module.ts`

```
import { APP_INITIALIZER, NgModule } from '@angular/core';
import { BrowserModule } from '@angular/platform-browser';
import { BrowserAnimationsModule } from '@angular/platform-browser/animations';
import { HTTP_INTERCEPTORS, HttpClientModule } from '@angular/common/http';
import { firstValueFrom } from 'rxjs';
```

```
import { AppRoutingModuleModule } from './app-routing.module';
import { AppComponent } from './app.component';
import { SharedModule } from './shared/shared.module';
import { AuthInterceptor } from './services/auth.interceptor';
import { AuthService } from './services/auth.service';

import { NavbarComponent } from './components/navbar/navbar.component';
import { HomeComponent } from './components/home/home.component';
import { DashboardComponent } from './components/dashboard/dashboard.component';
import { ErrorComponent } from './components/error/error.component';
import { ConfirmDialogComponent } from './components/confirm-dialog/confirm-dialog.component';
import { FooterComponent } from './components/footer/footer.component';

/**
 * Only what every visitor needs is loaded up front: the navbar, home, dashboard, error page and the dialog.
 * Login, admin, customer and course-detail pages are lazy modules (see app-routing.module.ts).
 *
 * @author Team Lead
 */
@NgModule({
  declarations: [AppComponent, NavbarComponent, HomeComponent, DashboardComponent, ErrorComponent, ConfirmDialogComponent, FooterComponent],
  imports: [BrowserModule, BrowserAnimationsModule, HttpClientModule, SharedModule, AppRoutingModuleModule],
  providers: [
    // the token is added to API calls here and nowhere else (guideline 24)
    { provide: HTTP_INTERCEPTORS, useClass: AuthInterceptor, multi: true },
    // after a page reload, bring the session back (saved token or refresh cookie) before the first page renders
    {
      provide: APP_INITIALIZER,
      multi: true,
      deps: [AuthService],
      useFactory: (auth: AuthService) => () => firstValueFrom(auth.restoreSession())
    }
  ],
  bootstrap: [AppComponent]
})
export class AppModule {}
```

angularapp/src/app/app.component.html

```
<app-navbar></app-navbar>
<router-outlet></router-outlet>
<app-footer></app-footer>
```

angularapp/src/app/app.component.css

```
:host { display: flex; flex-direction: column; min-height: 100vh; }
app-footer { margin-top: auto; }
```

F7. Registration rules

Rules: username has no spaces; email is a name, @, a domain, a dot and at least 2 letters; password has 1 upper, 1 lower, 1 number, 1 special character, no spaces, 8 to 72 characters (a live checklist shows what is still missing); mobile has 10 digits and starts with 6, 7, 8 or 9.

angularapp/src/app/components/registration/registration.component.ts

```
import { Component } from '@angular/core';
import { AbstractControl, FormBuilder, FormGroup, FormGroupDirective, NgForm, ValidationErrors, Validators } from '@angular/forms';
import { ErrorStateMatcher } from '@angular/material/core';
import { Router } from '@angular/router';
import { AuthService } from '../../services/auth.service';

/** Username: no spaces anywhere. */
function noSpaces(control: AbstractControl): ValidationErrors | null {
  return /\s/.test(control.value || '') ? { spaces: true } : null;
}

/** Password: one upper case, one lower case, one number, one special character, and no spaces. */
export const PASSWORD_RULES = [
  { key: 'length', label: '8 to 72 characters', test: (v: string) => v.length >= 8 && v.length <= 72 },
  { key: 'upper', label: 'One upper case letter', test: (v: string) => /[A-Z]/.test(v) },
  { key: 'lower', label: 'One lower case letter', test: (v: string) => /[a-z]/.test(v) },
  { key: 'digit', label: 'One number', test: (v: string) => /[0-9]/.test(v) },
  { key: 'special', label: 'One special character (for example @ # $ % !)', test: (v: string) => /[^\A-Za-z0-9\s]/.test(v) },
  { key: 'space', label: 'No spaces', test: (v: string) => v.length > 0 && !/\s/.test(v) }
];

function strongPassword(control: AbstractControl): ValidationErrors | null {
  const v: string = control.value || '';
  return v && PASSWORD_RULES.some(r => !r.test(v)) ? { weak: true } : null;
}

/** Email: name, one @, then a domain with a dot followed by at least 2 letters (a@b.com works, a@b does not). */
const EMAIL_PATTERN = /^[A-Za-z0-9-.%+\-]@[A-Za-z0-9\-\-]+\([\.A-Za-z0-9\-\-]+\)*\.[A-Za-z]{2,}$/;

/** Mobile: 10 digits, the first one is 6, 7, 8 or 9. */
const MOBILE_PATTERN = /^[6-9][0-9]{9}$/;

function passwordsMatch(group: AbstractControl): ValidationErrors | null {
  const p = group.get('password')?.value;
  const c = group.get('confirmPassword')?.value;
  return p && c && p !== c ? { mismatch: true } : null;
}

/** Shows the "Password do not match" error on the confirm field, even though the rule lives on the whole form. */
class ConfirmMatcher implements ErrorStateMatcher {
  isErrorState(control: AbstractControl | null, form: FormGroupDirective | NgForm | null): boolean {
    return !!control && (control.touched || control.dirty) && (control.invalid || !!form?.form.hasError('mismatch'));
  }
}
```

```

}

@Component({
  selector: 'app-registration',
  templateUrl: './registration.component.html',
  styleUrls: ['./registration.component.css']
})
export class RegistrationComponent {
  registerForm: FormGroup;
  errorMessage = '';
  loading = false;
  showPassword = false;
  passwordFocused = false;
  confirmMatcher = new ConfirmMatcher();

  constructor(private fb: FormBuilder, private auth: AuthService, private router: Router) {
    this.registerForm = this.fb.group({
      username: ['', [Validators.required, Validators.minLength(3), noSpaces]],
      email: ['', [Validators.required, Validators.pattern(EMAIL_PATTERN)]],
      password: ['', [Validators.required, strongPassword]],
      confirmPassword: ['', [Validators.required]],
      mobileNumber: ['', [Validators.required, Validators.pattern(MOBILE_PATTERN)]],
      role: ['Customer'] // fixed: public sign-up always creates a customer
    }, { validators: passwordsMatch });
  }

  get f() { return this.registerForm.controls; }

  /** Live checklist under the password field. */
  get passwordChecks() {
    const v: string = this.f['password'].value || '';
    return PASSWORD_RULES.map(r => ({ label: r.label, ok: r.test(v) }));
  }

  onSubmit(): void {
    if (this.registerForm.invalid || this.loading) { return; }
    this.loading = true;
    this.errorMessage = '';
    const { confirmPassword, ...user } = this.registerForm.value;
    this.auth.register(user).subscribe({
      next: () => { this.loading = false; this.router.navigate(['/login']); },
      error: (err) => {
        this.loading = false;
        if (err.status === 409) {
          this.errorMessage = 'A user with this email already exists';
        } else if (err.status === 429) {
          this.errorMessage = 'Too many attempts. Please wait a minute and try again.';
        } else if (err.status === 400) {
          this.errorMessage = err.error?.message || 'Please check the entered details';
        } else {
          this.errorMessage = 'Registration failed. Please try again later.';
        }
      }
    });
  }
}

```

angularapp/src/app/components/registration/registration.component.html

```

<div class="auth">
  <section class="auth-hero">
    <h1>Start learning today.</h1>
    <p>Create a free customer account to browse courses, enroll and review them.</p>
    <ul>
      <li><mat-icon>person_add</mat-icon>One account, one minute to set up</li>
      <li><mat-icon>lock</mat-icon>Your password is stored securely</li>
      <li><mat-icon>devices</mat-icon>Works on desktop and mobile</li>
    </ul>
  </section>

  <section class="auth-form">
    <div class="form-card">
      <h2>Registration</h2>
      <p class="lead">Fields marked * are required.</p>
      <form [formGroup]="registerForm" (ngSubmit)="onSubmit()" novalidate>
        <mat-form-field appearance="outline">
          <mat-label>Username</mat-label>
          <input matInput id="username" type="text" formControlName="username" autocomplete="username" />
          <mat-error *ngIf="f['username'].errors?.['required']">Username is required</mat-error>
          <mat-error *ngIf="f['username'].errors?.['minlength']">Username needs at least 3 characters</mat-error>
          <mat-error *ngIf="f['username'].errors?.['spaces']">Username cannot contain spaces</mat-error>
        </mat-form-field>

        <mat-form-field appearance="outline">
          <mat-label>Email</mat-label>
          <input matInput id="email" type="email" formControlName="email" autocomplete="email" />
          <mat-error *ngIf="f['email'].errors?.['required']">Email is required</mat-error>
          <mat-error *ngIf="f['email'].errors?.['pattern']">Enter a valid email, for example name&#64;example.com</mat-error>
        </mat-form-field>

        <mat-form-field appearance="outline">
          <mat-label>Password</mat-label>
          <input matInput id="password" [type]="showPassword ? 'text' : 'password'" formControlName="password" autocomplete="new-password"
(focus)="passwordFocused = true" />
          <button mat-icon-button matSuffix type="button" (click)="showPassword = !showPassword" [attr.aria-label]="showPassword ? 'Hide password' : 'Show password'">
            <mat-icon>{{ showPassword ? 'visibility_off' : 'visibility' }}</mat-icon>
          </button>
          <mat-error *ngIf="f['password'].errors?.['required']">Password is required</mat-error>
          <mat-error *ngIf="f['password'].errors?.['weak']">Password does not meet all the rules below</mat-error>
        </mat-form-field>
        <ul class="pw-rules" id="passwordRules" aria-label="Password rules" *ngIf="passwordFocused || f['password'].value">
          <li *ngFor="let c of passwordChecks" [class.ok]="c.ok">
            <mat-icon aria-hidden="true">{{ c.ok ? 'check_circle' : 'radio_button_unchecked' }}</mat-icon>{{ c.label }}
          </li>
        </ul>
      </form>
    </div>
  </section>

```

```

<mat-form-field appearance="outline">
  <mat-label>Confirm Password</mat-label>
  <input matInput id="confirmPassword" [type]="showPassword ? 'text' : 'password'" formControlName="confirmPassword" autocomplete="new-password"
[errorStateMatcher]="confirmMatcher" />
  <mat-error *ngIf="f['confirmPassword'].errors?.['required']">Confirm password is required</mat-error>
  <mat-error *ngIf="!f['confirmPassword'].errors?.['required'] && registerForm.errors?.['mismatch']">Password do not match</mat-error>
</mat-form-field>

<mat-form-field appearance="outline">
  <mat-label>Mobile Number</mat-label>
  <input matInput id="mobileNumber" type="text" inputmode="numeric" formControlName="mobileNumber" maxLength="10" />
  <mat-error *ngIf="f['mobileNumber'].errors?.['required']">Mobile number is required</mat-error>
  <mat-error *ngIf="f['mobileNumber'].errors?.['pattern']">Enter a 10 digit mobile number starting with 6, 7, 8 or 9</mat-error>
</mat-form-field>

<div class="form-error" role="alert" *ngIf="errorMessage"><mat-icon>error_outline</mat-icon>{{ errorMessage }}</div>
<button mat-flat-button color="primary" type="submit" id="registerButton" [disabled]="registerForm.invalid || loading">
  {{ loading ? 'Creating account...' : 'Register' }}
</button>
</form>
<p class="alt">Already have an account? <a routerLink="/login">Login</a></p>
</div>
</section>
</div>

```

F8. Add and edit course: browse an image

The link box is gone. The admin clicks **Browse image**, sees a preview at once, the file is uploaded, and the returned URL is kept in the same `courseImageUrl` form field. The submit button waits while the upload runs. Wrong type or over 5 MB shows a message.

angularapp/src/app/components/add-course/add-course.component.ts

```

import { Component, OnDestroy } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { MatSnackBar } from '@angular/material/snack-bar';
import { CourseService } from '../../../services/course.service';
import { ImageUploadService } from '../../../services/image-upload.service';

@Component({
  selector: 'app-add-course',
  templateUrl: './add-course.component.html',
  styleUrls: ['./add-course.component.css']
})
export class AddCourseComponent implements OnDestroy {
  courseForm: FormGroup;
  errorMessage = '';
  loading = false;
  imgFailed = false;
  uploading = false;
  uploadError = '';
  previewUrl = '';

  constructor(private fb: FormBuilder, private courseService: CourseService, private router: Router, private snack: MatSnackBar, private images:
ImageUploadService) {
    this.courseForm = this.fb.group({
      courseType: ['', [Validators.required]],
      courseImageUrl: ['', [Validators.required, Validators.pattern(/^https?:\/\/.+$/)]],
      courseDetails: ['', [Validators.required]],
      coursePrice: [null, [Validators.required, Validators.min(0)]]
    });
  }

  get f() { return this.courseForm.controls; }

  /** The admin picked a file: check it, show a preview at once, upload it, and keep the returned URL in the form. */
  onFileSelected(event: Event): void {
    const input = event.target as HTMLInputElement;
    const file = input.files?.[0];
    input.value = ''; // lets the same file be chosen again
    if (!file) { return; }
    this.uploadError = '';
    const problem = this.images.validate(file);
    if (problem) { this.uploadError = problem; return; }
    this.clearPreview();
    this.previewUrl = URL.createObjectURL(file);
    this.imgFailed = false;
    this.uploading = true;
    this.f['courseImageUrl'].markAsTouched();
    this.images.upload(file).subscribe({
      next: (url) => { this.f['courseImageUrl'].setValue(url); this.uploading = false; },
      error: (err) => {
        this.uploading = false;
        this.clearPreview();
        this.uploadError = err.error?.message || 'Could not upload the image. Please try again.';
      }
    });
  }

  private clearPreview(): void {
    if (this.previewUrl) { URL.revokeObjectURL(this.previewUrl); }
    this.previewUrl = '';
  }

  ngOnDestroy(): void {
    this.clearPreview();
  }

  onSubmit(): void {
    if (this.courseForm.invalid || this.loading) { return; }
    this.loading = true;
    this.errorMessage = '';
    this.courseService.addCourse(this.courseForm.value).subscribe({
      next: () => { this.loading = false; this.snack.open('Course added', 'OK', { duration: 3000 }); this.router.navigate(['/view-courses']); },

```

```

    error: () => { this.loading = false; this.errorMessage = 'Could not add the course. Please try again.'; }
  });
}
}

```

angularapp/src/app/components/add-course/add-course.component.html

```

<div class="page">
  <div class="form-card">
    <h2>Add New Course</h2>
    <p class="lead">Fields marked * are required.</p>
    <form [formGroup]="courseForm" (ngSubmit)="onSubmit()" novalidate>
      <mat-form-field appearance="outline">
        <mat-label>Course Type</mat-label>
        <input matInput id="courseType" type="text" formControlName="courseType" />
        <mat-error *ngIf="f['courseType'].errors?.['required']">Course type is required</mat-error>
      </mat-form-field>

      <!-- the image is chosen from the computer; the server saves it and returns its URL, which goes into courseImageUrl -->
      <div class="upload">
        <input #fileInput type="file" id="courseImageFile" hidden accept="image/png,image/jpeg,image/gif,image/webp" (change)="onFileSelected($event)" />
        <div class="img-preview">
          <img *ngIf="(previewUrl || courseForm.value.courseImageUrl) && !imgFailed"
            [src]="previewUrl || courseForm.value.courseImageUrl"
            (error)="imgFailed = true"
            (load)="imgFailed = false"
            alt="Course image preview">
          <span *ngIf="!previewUrl && !courseForm.value.courseImageUrl">Image preview appears here</span>
          <span *ngIf="!previewUrl && courseForm.value.courseImageUrl && imgFailed">This image could not be loaded. Choose another image.</span>
        </div>
        <div class="upload-row">
          <button mat-stroked-button color="primary" type="button" id="browseImage" [disabled]="uploading" (click)="fileInput.click()">
            <mat-icon>upload</mat-icon>{{ uploading ? 'Uploading...' : (courseForm.value.courseImageUrl ? 'Change image' : 'Browse image') }}
          </button>
          <span class="upload-hint">PNG, JPG, GIF or WEBP, up to 5 MB</span>
        </div>
        <div class="form-error small" role="alert" id="uploadError" *ngIf="uploadError">{{ uploadError }}</div>
        <div class="form-error small" *ngIf="!uploadError && !uploading && f['courseImageUrl'].touched && f['courseImageUrl'].errors?.['required']">Course image
is required</div>
      </div>

      <mat-form-field appearance="outline">
        <mat-label>Course Details</mat-label>
        <textarea matInput id="courseDetails" rows="3" formControlName="courseDetails"></textarea>
        <mat-error *ngIf="f['courseDetails'].errors?.['required']">Course details are required</mat-error>
      </mat-form-field>

      <mat-form-field appearance="outline">
        <mat-label>Price</mat-label>
        <span matTextPrefix>&#8377;&nbsp;</span>
        <input matInput id="coursePrice" type="number" min="0" formControlName="coursePrice" />
        <mat-error *ngIf="f['coursePrice'].errors?.['required']">Price is required</mat-error>
        <mat-error *ngIf="f['coursePrice'].errors?.['min']">Price cannot be negative</mat-error>
      </mat-form-field>

      <div class="form-error" role="alert" *ngIf="errorMessage"><mat-icon>error_outline</mat-icon>{{ errorMessage }}</div>
      <div class="form-actions">
        <button mat-flat-button color="primary" type="submit" id="addCourseButton" [disabled]="courseForm.invalid || loading || uploading">Add Course</button>
      </div>
    </form>
  </div>
</div>

```

angularapp/src/app/components/edit-course/edit-course.component.ts

```

import { Component, OnDestroy, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { ActivatedRoute, Router } from '@angular/router';
import { MatSnackBar } from '@angular/material/snack-bar';
import { CourseService } from '../../../services/course.service';
import { ImageUploadService } from '../../../services/image-upload.service';

@Component({
  selector: 'app-edit-course',
  templateUrl: './edit-course.component.html',
  styleUrls: ['./edit-course.component.css']
})
export class EditCourseComponent implements OnInit, OnDestroy {
  courseForm: FormGroup;
  courseId = '';
  errorMessage = '';
  loading = false;
  imgFailed = false;
  uploading = false;
  uploadError = '';
  previewUrl = '';

  constructor(
    private fb: FormBuilder,
    private route: ActivatedRoute,
    private router: Router,
    private courseService: CourseService,
    private snack: MatSnackBar,
    private images: ImageUploadService
  ) {
    this.courseForm = this.fb.group({
      courseType: ['', [Validators.required]],
      courseImageUrl: ['', [Validators.required, Validators.pattern(/^https?:\/\/.+\/)]],
      courseDetails: ['', [Validators.required]],
      coursePrice: [null, [Validators.required, Validators.min(0)]]
    });
  }

  get f() { return this.courseForm.controls; }

```

```

/** The admin picked a file: check it, show a preview at once, upload it, and keep the returned URL in the form. */
onFileSelected(event: Event): void {
  const input = event.target as HTMLInputElement;
  const file = input.files?.[0];
  input.value = ''; // lets the same file be chosen again
  if (!file) { return; }
  this.uploadError = '';
  const problem = this.images.validate(file);
  if (problem) { this.uploadError = problem; return; }
  this.clearPreview();
  this.previewUrl = URL.createObjectURL(file);
  this.imgFailed = false;
  this.uploading = true;
  this.f['courseImageUrl'].markAsTouched();
  this.images.upload(file).subscribe({
    next: (url) => { this.f['courseImageUrl'].set_value(url); this.uploading = false; },
    error: (err) => {
      this.uploading = false;
      this.clearPreview();
      this.uploadError = err.error?.message || 'Could not upload the image. Please try again.';
    }
  });
}

private clearPreview(): void {
  if (this.previewUrl) { URL.revokeObjectURL(this.previewUrl); }
  this.previewUrl = '';
}

ngOnDestroy(): void {
  this.clearPreview();
}

ngOnInit(): void {
  this.courseId = this.route.snapshot.paramMap.get('id') || '';
  this.courseService.getCourseById(this.courseId).subscribe({
    next: (course) => this.courseForm.patchValue(course),
    error: () => (this.errorMessage = 'Could not load this course.')
  });
}

onSubmit(): void {
  if (this.courseForm.invalid || this.loading) { return; }
  this.loading = true;
  this.courseService.updateCourse(this.courseId, this.courseForm.value).subscribe({
    next: () => { this.loading = false; this.snack.open('Course updated', 'OK', { duration: 3000 }); this.router.navigate(['/view-courses']); },
    error: () => { this.loading = false; this.errorMessage = 'Could not update the course. Please try again.'; }
  });
}

cancel(): void {
  this.router.navigate(['/view-courses']);
}
}

```

angularapp/src/app/components/edit-course/edit-course.component.html

```

<div class="page">
  <div class="form-card">
    <h2>Edit Course</h2>
    <p class="lead">Change any field and update the course.</p>
    <form [formGroup]="courseForm" (ngSubmit)="onSubmit()" novalidate>
      <mat-form-field appearance="outline">
        <mat-label>Course Type</mat-label>
        <input matInput id="courseType" type="text" formControlName="courseType" />
        <mat-error *ngIf="f['courseType'].errors?.['required']">Course type is required</mat-error>
      </mat-form-field>

      <!-- the image is chosen from the computer; the server saves it and returns its URL, which goes into courseImageUrl -->
      <div class="upload">
        <input #fileInput type="file" id="courseImageFile" hidden accept="image/png,image/jpeg,image/gif,image/webp" (change)="onFileSelected($event)" />
        <div class="img-preview">
          <img *ngIf="(previewUrl || courseForm.value.courseImageUrl) && !imgFailed"
              [src]="previewUrl || courseForm.value.courseImageUrl"
              (error)="imgFailed = true"
              (load)="imgFailed = false"
              alt="Course image preview">
          <span *ngIf="!previewUrl && !courseForm.value.courseImageUrl">Image preview appears here</span>
          <span *ngIf="!previewUrl && courseForm.value.courseImageUrl && imgFailed">This image could not be loaded. Choose another image.</span>
        </div>
        <div class="upload-row">
          <button mat-stroked-button color="primary" type="button" id="browseImage" [disabled]="uploading" (click)="fileInput.click()">
            <mat-icon>upload</mat-icon>{{ uploading ? 'Uploading...' : (courseForm.value.courseImageUrl ? 'Change image' : 'Browse image') }}
          </button>
          <span class="upload-hint">PNG, JPG, GIF or WEBP, up to 5 MB</span>
        </div>
        <div class="form-error small" role="alert" id="uploadError" *ngIf="uploadError">{{ uploadError }}</div>
        <div class="form-error small" *ngIf="!uploadError && !uploading && f['courseImageUrl'].touched && f['courseImageUrl'].errors?.['required']">Course image
is required</div>
      </div>

      <mat-form-field appearance="outline">
        <mat-label>Course Details</mat-label>
        <textarea matInput id="courseDetails" rows="3" formControlName="courseDetails"></textarea>
        <mat-error *ngIf="f['courseDetails'].errors?.['required']">Course details are required</mat-error>
      </mat-form-field>

      <mat-form-field appearance="outline">
        <mat-label>Price</mat-label>
        <span matTextPrefix>&#37;&#x2013;</span>
        <input matInput id="coursePrice" type="number" min="0" formControlName="coursePrice" />
        <mat-error *ngIf="f['coursePrice'].errors?.['required']">Price is required</mat-error>
        <mat-error *ngIf="f['coursePrice'].errors?.['min']">Price cannot be negative</mat-error>
      </mat-form-field>

      <div class="form-error" role="alert" *ngIf="errorMessage"><mat-icon>error_outline</mat-icon>{{ errorMessage }}</div>
      <div class="form-actions">

```

```

        <button mat-flat-button color="primary" type="submit" id="updateCourseButton" [disabled]="courseForm.invalid || loading || uploading">Update
Course</button>
        <button mat-stroked-button color="warn" type="button" id="cancelEdit" (click)="cancel()">Cancel</button>
      </div>
    </form>
  </div>
</div>

```

F9. Bought course: "Go to course"

On the catalog, a course the customer already bought shows an **Enrolled** badge and a **Go to course** button instead of Add to Cart. That button opens the course page, which now shows "You are enrolled" with **Go to my course** (your enrollments) and **Write a review**. On the same page a customer sees a delete icon on his own review.

angularapp/src/app/shared/course-card/course-card.component.ts

```

import { Component, EventEmitter, Input, Output } from '@angular/core';
import { Course } from '../../../models/course.model';
import { CourseRating } from '../../../models/course-rating.model';

/**
 * One course tile, used on the admin course list and the customer catalog.
 * It only shows data and raises events; the page decides what happens.
 */
* @author Amogh
*/
@Component({
  selector: 'app-course-card',
  templateUrl: './course-card.component.html',
  styleUrls: ['./course-card.component.css']
})
export class CourseCardComponent {
  @Input() course!: Course;
  @Input() rating?: CourseRating;
  /** 'customer' shows Add to Cart, 'admin' shows Edit and Delete. */
  @Input() mode: 'admin' | 'customer' = 'customer';
  @Input() inCart = false;
  /** True when the customer already bought this course: the tile shows "Go to course" instead of Add to Cart. */
  @Input() enrolled = false;
  @Output() addToCart = new EventEmitter<Course>();
  @Output() edit = new EventEmitter<Course>();
  @Output() remove = new EventEmitter<Course>();
}

```

angularapp/src/app/shared/course-card/course-card.component.html

```

<article class="card">
  <a class="cover" [routerLink]="['/course-details', course.courseId]" [attr.aria-label]="View details of ' + course.courseType">
    <img [src]="course.courseImageUrl" [alt]="course.courseType" loading="lazy" (error)="$(event.target).style.display='none'" />
    <span class="initial" aria-hidden="true">{{ course.courseType?.charAt(0) }}</span>
    <span class="enrolled-badge" *ngIf="mode === 'customer' && enrolled"><mat-icon>check_circle</mat-icon>Enrolled</span>
  </a>
  <div class="body">
    <h3><a [routerLink]="['/course-details', course.courseId]">{{ course.courseType }}</a></h3>
    <p>{{ course.courseDetails }}</p>
    <app-star-rating [value]="rating?.averageRating || 0" [count]="rating?.reviewCount"></app-star-rating>
    <span class="price">{{ course.coursePrice | currency:'INR': 'symbol':'1.0-2' }}</span>
  </div>
  <div class="foot" *ngIf="mode === 'customer' && enrolled">
    <a mat-flat-button color="primary" class="add-cart go-course" [routerLink]="['/course-details', course.courseId]"><mat-icon>play_circle</mat-icon>Go to
course</a>
  </div>
  <div class="foot" *ngIf="mode === 'customer' && !enrolled">
    <button mat-flat-button color="primary" class="add-cart" (click)="addToCart.emit(course)">
      <mat-icon>add_shopping_cart</mat-icon>{{ inCart ? 'Added' : 'Add to Cart' }}
    </button>
    <a mat-stroked-button [routerLink]="['/course-details', course.courseId]">Details</a>
  </div>
  <div class="foot" *ngIf="mode === 'admin'">
    <button mat-stroked-button color="primary" class="edit-btn" (click)="edit.emit(course)"><mat-icon>edit</mat-icon>Edit</button>
    <button mat-stroked-button color="warn" class="delete-btn" (click)="remove.emit(course)"><mat-icon>delete</mat-icon>Delete</button>
  </div>
</div>
</article>

```

angularapp/src/app/components/customer-view-courses/customer-view-courses.component.ts

```

import { Component, OnDestroy, OnInit } from '@angular/core';
import { ActivatedRoute } from '@angular/router';
import { PageEvent } from '@angular/material/paginator';
import { MatSnackBar } from '@angular/material/snack-bar';
import { Subscription } from 'rxjs';
import { PAGE_SIZE } from '../../../constant';
import { Course } from '../../../models/course.model';
import { CourseRating } from '../../../models/course-rating.model';
import { CourseSearchParams } from '../../../models/search-params.model';
import { AIService } from '../../../services/ai.service';
import { CartService } from '../../../services/cart.service';
import { CourseService } from '../../../services/course.service';
import { EnrollmentService } from '../../../services/enrollment.service';

/**
 * Course catalog for customers: keyword search, price filter, sorting, server-side paging,
 * and the AI (semantic) search box. Ratings come from the review summary API.
 */
* @author Amogh (catalog, filters, pagination), Sumit (AI search)
*/
@Component({
  selector: 'app-customer-view-courses',
  templateUrl: './customer-view-courses.component.html',

```

```

styleUrls: ['./customer-view-courses.component.css']
})
export class CustomerViewCoursesComponent implements OnInit, OnDestroy {
  courses: Course[] = [];
  ratings: { [courseType: string]: CourseRating } = {};
  inCart = new Set<number>();
  enrolled = new Set<number>(); // courses this customer already bought
  errorMessage = '';
  toastMessage = '';
  loaded = false;

  // filters (server-side)
  filters: CourseSearchParams = { keyword: '', minPrice: null, maxPrice: null, sort: 'newest', page: 0, size: PAGE_SIZE };
  totalElements = 0;
  pageSize = PAGE_SIZE;
  priceError = '';

  // AI search state
  searchText = ''; // the one search box: used by Search (keyword) and AI Search
  aiActive = false;
  aiLoading = false;
  aiError = '';
  activeQuery = '';

  private sub?: Subscription;

  constructor(
    private route: ActivatedRoute,
    private courseService: CourseService,
    private cartService: CartService,
    private aiService: AiService,
    private snack: MatSnackBar,
    private enrollment: EnrollmentService
  ) {}

  ngOnInit(): void {
    this.enrollment.enrolledCourseIds().subscribe(ids => (this.enrolled = ids));
    this.cartService.getReviewSummary().subscribe({
      next: (list) => (list || []).forEach(r => (this.ratings[r.courseType || ''] = r)),
      error: () => (this.ratings = {})
    });
    // the navbar search box puts its text in ?keyword=
    this.sub = this.route.queryParamMap.subscribe(params => {
      this.filters.keyword = params.get('keyword') || '';
      this.searchText = this.filters.keyword;
      this.filters.page = 0;
      this.aiActive = false;
      this.load();
    });
  }

  ngOnDestroy(): void {
    this.sub?.unsubscribe();
  }

  get hasFilters(): boolean {
    return !(this.filters.keyword || this.filters.minPrice != null || this.filters.maxPrice != null);
  }

  load(): void {
    this.errorMessage = '';
    this.courseService.searchCourses(this.filters).subscribe({
      next: (res) => {
        this.courses = res.content || [];
        this.totalElements = res.totalElements || 0;
        this.loaded = true;
      },
      error: () => { this.errorMessage = 'Could not load courses.'; this.loaded = true; }
    });
  }

  /** Plain keyword search (the Search button or Enter in the search box). */
  keywordSearch(): void {
    this.filters.keyword = this.searchText.trim();
    this.applyFilters();
  }

  applyFilters(): void {
    const { minPrice, maxPrice } = this.filters;
    this.priceError = minPrice != null && maxPrice != null && minPrice > maxPrice ? 'Minimum price cannot be more than maximum price' : '';
    if (this.priceError) { return; }
    this.filters.page = 0;
    this.aiActive = false;
    this.load();
  }

  clearFilters(): void {
    this.filters = { keyword: '', minPrice: null, maxPrice: null, sort: 'newest', page: 0, size: PAGE_SIZE };
    this.searchText = '';
    this.aiActive = false;
    this.priceError = '';
    this.load();
  }

  onPage(event: PageEvent): void {
    this.filters.page = event.pageIndex;
    this.load();
    window.scrollTo({ top: 0, behavior: 'smooth' });
  }

  ratingFor(course: Course): CourseRating | undefined {
    return this.ratings[course.courseType || ''];
  }

  aiSearch(): void {
    const query = this.searchText.trim();
    if (!query || this.aiLoading) { return; }
    this.aiLoading = true;
    this.aiError = '';
    this.aiService.searchCourses(query).subscribe({

```

```

    next: (results) => {
      this.courses = results;
      this.aiActive = true;
      this.activeQuery = query;
      this.aiLoading = false;
      this.loaded = true;
    },
    error: (err) => {
      this.aiLoading = false;
      this.aiError = err.status === 429
        ? (err.error?.message || 'You have reached the AI search limit. Please try again later.')
        : 'AI search failed. Please try again.';
    }
  });
}

clearAiSearch(): void {
  this.searchText = this.filters.keyword || '';
  this.activeQuery = '';
  this.aiActive = false;
  this.aiError = '';
  this.load();
}

addToCart(course: Course): void {
  if (course.courseId != null && this.enrolled.has(course.courseId)) {
    this.snack.open('You are already enrolled in this course', 'OK', { duration: 2500 });
    return;
  }
  this.cartService.addToCart(course).subscribe({
    next: () => {
      if (course.courseId != null) { this.inCart.add(course.courseId); }
      this.toastMessage = `${course.courseType} added to your cart`;
      this.snack.open(this.toastMessage, 'OK', { duration: 2500 });
    },
    error: () => {
      this.toastMessage = '';
      this.errorMessage = 'Could not add the course to your cart.';
    }
  });
}
}
}

```

angularapp/src/app/components/customer-view-courses/customer-view-courses.component.html

```

<section class="top">
  <div class="top-inner">
    <h2>Available Courses</h2>
    <p>Search by name, or use AI Search to describe what you want in plain words, for example "cheap beginner course".</p>
    <form class="ai-search" (ngSubmit)="keywordSearch()">
      <mat-form-field appearance="outline" class="ai-field" subscriptSizing="dynamic">
        <mat-label>Search courses</mat-label>
        <mat-icon matPrefix>search</mat-icon>
        <input matInput id="aiSearchInput" name="searchText" type="text" [(ngModel)]="searchText" maxlength="200"
          placeholder="Try: 'java' or, with AI Search, 'cheap beginner course'" />
      </mat-form-field>
      <button mat-flat-button color="primary" type="submit" id="searchButton" class="solid">Search</button>
      <button mat-flat-button color="accent" type="button" id="aiSearchButton" (click)="aiSearch()" [disabled]="aiLoading || !searchText.trim()">
        <mat-icon>auto_awesome</mat-icon>{{ aiLoading ? 'Searching...' : 'AI Search' }}
      </button>
      <button mat-stroked-button type="button" class="ghost" id="clearAiButton" *ngIf="aiActive" (click)="clearAiSearch()">Clear AI Search</button>
    </form>
  </div>
</section>

<div class="page">
  <p class="form-note" *ngIf="aiActive">Showing AI-matched results for: '{{ activeQuery }}'</p>
  <div class="form-error" role="alert" *ngIf="aiError"><mat-icon>error_outline</mat-icon>{{ aiError }}</div>
  <div class="form-error" role="alert" *ngIf="errorMessage"><mat-icon>error_outline</mat-icon>{{ errorMessage }}</div>

  <div class="layout">
    <aside class="filters" *ngIf="!aiActive">
      <h3>Filter</h3>
      <form (ngSubmit)="applyFilters()">
        <div class="price-row">
          <mat-form-field appearance="outline" subscriptSizing="dynamic">
            <mat-label>Min ₹</mat-label>
            <input matInput id="minPrice" name="minPrice" type="number" min="0" [(ngModel)]="filters.minPrice" />
          </mat-form-field>
          <mat-form-field appearance="outline" subscriptSizing="dynamic">
            <mat-label>Max ₹</mat-label>
            <input matInput id="maxPrice" name="maxPrice" type="number" min="0" [(ngModel)]="filters.maxPrice" />
          </mat-form-field>
        </div>
        <div class="form-error small" *ngIf="priceError">{{ priceError }}</div>
        <mat-form-field appearance="outline" subscriptSizing="dynamic">
          <mat-label>Sort by</mat-label>
          <mat-select id="sort" name="sort" [(ngModel)]="filters.sort" (selectionChange)="applyFilters()">
            <mat-option value="newest">Newest</mat-option>
            <mat-option value="priceAsc">Price: low to high</mat-option>
            <mat-option value="priceDesc">Price: high to low</mat-option>
            <mat-option value="name">Name</mat-option>
          </mat-select>
        </mat-form-field>
        <button mat-flat-button color="primary" type="submit" id="applyFilters">Apply filters</button>
        <button mat-button type="button" id="clearFilters" *ngIf="hasFilters" (click)="clearFilters()">Clear filters</button>
      </form>
    </aside>

    <section class="results">
      <p class="count" id="resultCount" *ngIf="loaded && courses.length > 0">
        {{ aiActive ? courses.length : totalElements }} course{{ aiActive ? courses.length : totalElements } === 1 ? '' : 's' }} found
      </p>
      <div class="empty" *ngIf="loaded && !errorMessage && courses.length === 0"><mat-icon>search_off</mat-icon><h3>No courses found</h3><p>Try different words or
        a wider price range.</p></div>
      <div class="course-grid">
        <app-course-card *ngFor="let course of courses" mode="customer" [course]="course" [rating]="ratingFor(course)"

```

```

        [inCart]="!!course.courseId && inCart.has(course.courseId)"
        [enrolled]="!!course.courseId && enrolled.has(course.courseId)" (addToCart)="addToCart($event)"></app-course-card>
    </div>
    <mat-paginator *ngIf="!aiActive && totalElements > pageSize" [length]="totalElements" [pageSize]="pageSize"
        [pageIndex]="filters.page || 0" [hidePageSize]="true" (page)="onPage($event)" showFirstLastButtons></mat-paginator>
</section>
</div>
</div>

```

angularapp/src/app/components/course-details/course-details.component.ts

```

import { Component, OnInit } from '@angular/core';
import { ActivatedRoute, Router } from '@angular/router';
import { MatDialog } from '@angular/material/dialog';
import { MatSnackBar } from '@angular/material/snack-bar';
import { Course } from '../../../models/course.model';
import { Review } from '../../../models/review.model';
import { AuthService } from '../../../services/auth.service';
import { CartService } from '../../../services/cart.service';
import { ConfirmDialogComponent } from '../../../confirm-dialog/confirm-dialog.component';
import { CourseService } from '../../../services/course.service';
import { EnrollmentService } from '../../../services/enrollment.service';
import { ReviewService } from '../../../services/review.service';
import { ROLES } from '../../../constant';

/**
 * Course details page: summary band, price card with Add to Cart / Enroll, and the reviews of the course.
 */
* @author Amogh (course details), Sumit (reviews)
*/
@Component({
  selector: 'app-course-details',
  templateUrl: './course-details.component.html',
  styleUrls: ['./course-details.component.css']
})
export class CourseDetailsComponent implements OnInit {
  course?: Course;
  reviews: Review[] = [];
  average = 0;
  errorMessage = '';
  loaded = false;
  role: string | null;
  enrolled = false; // the customer already bought this course
  private myCustomerId = Number(localStorage.getItem('customerId'));

  constructor(
    private route: ActivatedRoute,
    private router: Router,
    private courseService: CourseService,
    private cartService: CartService,
    private auth: AuthService,
    private snack: MatSnackBar,
    private dialog: MatDialog,
    private enrollment: EnrollmentService,
    private reviewService: ReviewService
  ) {
    this.role = this.auth.getRole();
  }

  get isCustomer(): boolean {
    return this.role === ROLES.CUSTOMER;
  }

  ngOnInit(): void {
    const id = this.route.snapshot.paramMap.get('id') || '';
    this.courseService.getCourseById(id).subscribe({
      next: (course) => {
        this.course = course;
        this.loaded = true;
        this.loadReviews(course.courseType || '');
        if (this.isCustomer && course.courseId !== null) {
          this.enrollment.enrolledCourseIds().subscribe(ids => (this.enrolled = ids.has(course.courseId as number)));
        }
      },
      error: () => { this.errorMessage = 'This course could not be found.'; this.loaded = true; }
    });
  }

  private loadReviews(courseType: string): void {
    this.cartService.getReviewsByCourse(courseType).subscribe({
      // 204 No Content arrives as a null body
      next: (data) => {
        this.reviews = data || [];
        const total = this.reviews.reduce((sum, r) => sum + (r.rating || 0), 0);
        this.average = this.reviews.length ? total / this.reviews.length : 0;
      },
      error: () => (this.reviews = [])
    });
  }

  addToCart(enrollNow = false): void {
    if (!this.course || this.enrolled) { return; }
    this.cartService.addToCart(this.course).subscribe({
      next: () => {
        if (enrollNow) { this.router.navigate(['/my-cart']); return; }
        this.snack.open(`${this.course?.courseType} added to your cart`, 'OK', { duration: 2500 });
      },
      error: () => (this.errorMessage = 'Could not add the course to your cart.')
    });
  }

  /** True when this review was written by the logged-in customer. */
  isMine(review: Review): boolean {
    return this.isCustomer && !!this.myCustomerId && review.customer?.customerId === this.myCustomerId;
  }

  deleteReview(review: Review): void {

```

```

    this.dialog.open(ConfirmDialogComponent, {
      data: { title: 'Delete your review?', message: 'This removes your review of this course.', confirmText: 'Delete', cancelText: 'Cancel', icon: 'delete',
danger: true }
    }).afterClosed().subscribe(yes => {
      if (!yes || review.reviewId == null) { return; }
      this.reviewService.deleteMyReview(review.reviewId).subscribe({
        next: () => {
          this.snack.open('Review deleted', 'OK', { duration: 2500 });
          this.loadReviews(this.course?.courseType || '');
        },
        error: () => (this.errorMessage = 'Could not delete the review.')
      });
    });
  }

  back(): void {
    this.router.navigate([this.role === ROLES.ADMIN ? '/view-courses' : '/customer-view-courses']);
  }
}

```

angularapp/src/app/components/course-details/course-details.component.html

```

<div class="band" *ngIf="course">
  <div class="band-inner">
    <button mat-button class="back" (click)="back()"><mat-icon>arrow_back</mat-icon>All courses</button>
    <h1 id="courseTitle">{{ course.courseType }}</h1>
    <p class="lead">{{ course.courseDetails }}</p>
    <app-star-rating [value]="average" [count]="reviews.length"></app-star-rating>
  </div>
</div>

<div class="page" *ngIf="!course && loaded">
  <div class="empty"><mat-icon>search_off</mat-icon><h3>Course not found</h3><p>{{ errorMessage }}</p>
  <button mat-flat-button color="primary" (click)="back()">Back to courses</button></div>
</div>

<div class="page layout" *ngIf="course">
  <main>
    <section class="block">
      <h2>About this course</h2>
      <p>{{ course.courseDetails }}</p>
      <ul class="points">
        <li><mat-icon>all_inclusive</mat-icon>Learn at your own pace, on any device</li>
        <li><mat-icon>workspace_premium</mat-icon>Skills you can use at work and in studies</li>
        <li><mat-icon>event_available</mat-icon>Enroll once, access from your enrollments</li>
      </ul>
    </section>

    <section class="block" id="reviews">
      <h2>Learner reviews</h2>
      <div class="empty small" *ngIf="reviews.length === 0"><p>No reviews yet. Enrolled learners can add the first one.</p></div>
      <article class="review" *ngFor="let r of reviews">
        <div class="avatar" aria-hidden="true">{{ r.customer?.customerName?.charAt(0) || '?' }}</div>
        <div>
          <strong>{{ r.customer?.customerName || 'Learner' }}</strong>
          <app-star-rating [value]="r.rating || 0" [count]="1"></app-star-rating>
          <p>{{ r.body }}</p>
          <small class="muted">{{ r.dateCreated | date:'mediumDate' }}</small>
        </div>
        <button mat-icon-button color="warn" type="button" class="del-review" *ngIf="isMine(r)" (click)="deleteReview(r)" aria-label="Delete my review"
title="Delete my review"><mat-icon>delete</mat-icon></button>
      </article>
    </section>
  </main>

  <aside class="buy">
    <div class="preview">
      <img *ngIf="course.courseImageUrl" [src]="course.courseImageUrl" [alt]="course.courseType" (error)="$any($event.target).style.display='none'" />
      <span class="initial" aria-hidden="true">{{ course.courseType?.charAt(0) }}</span>
    </div>
    <div class="buy-body">
      <div class="price">{{ course.coursePrice | currency:'INR': 'symbol': '1.0-2' }}</div>
      <div class="form-error" role="alert" *ngIf="errorMessage"><mat-icon>error_outline</mat-icon>{{ errorMessage }}</div>
      <div class="enrolled-box" *ngIf="isCustomer && enrolled">
        <div class="enrolled-status"><mat-icon>check_circle</mat-icon><div><strong>You are enrolled</strong><span>This course is in your enrollments.</span></div>
      </div>
      <a mat-flat-button color="primary" class="wide" id="goToCourse" routerLink="/my-orders"><mat-icon>play_circle</mat-icon>Go to my course</a>
      <a mat-stroked-button color="primary" class="wide" id="writeReview" routerLink="/add-review"><mat-icon>rate_review</mat-icon>Write a review</a>
    </div>
    <ng-container *ngIf="isCustomer && !enrolled">
      <button mat-flat-button color="primary" class="wide" id="enrollFromDetails" (click)="addToCart(true)">Enroll now</button>
      <button mat-stroked-button color="primary" class="wide" id="addToCartFromDetails" (click)="addToCart()"><mat-icon>add_shopping_cart</mat-icon>Add to
Cart</button>
    </ng-container>
    <a mat-stroked-button color="primary" class="wide" *ngIf="role === 'ADMIN'" [routerLink]="['/edit-course', course.courseId]"><mat-icon>edit</mat-icon>Edit
course</a>
  </div>
</div>

```

F10. Cart and payment

If a bought course is already in the cart (for example it was added earlier), the cart shows an "Already enrolled" tag, a warning, and disables **Enroll Now** until it is removed. If the server refuses a purchase (409), the payment page shows its message instead of a generic one.

angularapp/src/app/components/my-cart/my-cart.component.ts

```

import { Component, OnInit } from '@angular/core';
import { Router } from '@angular/router';
import { Course } from '../../../models/course.model';
import { CartService } from '../../../services/cart.service';
import { EnrollmentService } from '../../../services/enrollment.service';

```

```

@Component({
  selector: 'app-my-cart',
  templateUrl: './my-cart.component.html',
  styleUrls: ['./my-cart.component.css']
})
export class MyCartComponent implements OnInit {
  courses: Course[] = [];
  totalAmount = 0;
  errorMessage = '';
  loaded = false;
  owned = new Set<number>(); // courses already bought

  constructor(private cartService: CartService, private router: Router, private enrollment: EnrollmentService) {}

  /** Cart items the customer already bought. They must be removed before enrolling. */
  get ownedInCart(): Course[] {
    return this.courses.filter(c => c.courseId != null && this.owned.has(c.courseId));
  }

  ngOnInit(): void {
    this.enrollment.enrolledCourseIds().subscribe(ids => (this.owned = ids));
    this.load();
  }

  load(): void {
    this.cartService.getAllCoursesFromCart().subscribe({
      next: (cart) => {
        this.courses = cart.courses || [];
        this.totalAmount = cart.totalAmount || 0;
        this.loaded = true;
      },
      error: (err) => {
        // 404 simply means the customer has not added anything yet
        this.courses = [];
        this.totalAmount = 0;
        this.loaded = true;
        if (err.status !== 404) { this.errorMessage = 'Could not load your cart.'; }
      }
    });
  }

  remove(course: Course): void {
    this.cartService.removeCoursesFromCart(course.courseId).subscribe({
      next: () => this.load(),
      error: () => (this.errorMessage = 'Could not remove the course.')
    });
  }

  enrollNow(): void {
    if (this.courses.length > 0 && this.ownedInCart.length === 0) { this.router.navigate(['/place-order']); }
  }
}

```

angularapp/src/app/components/my-cart/my-cart.component.html

```

<div class="page">
  <div class="page-head"><div><h2>My Cart</h2><p>Review your courses, then enroll.</p></div></div>
  <div class="form-error" role="alert" *ngIf="errorMessage"><mat-icon>error_outline</mat-icon>{{ errorMessage }}</div>
  <div class="empty" *ngIf="loaded && courses.length === 0">
    <mat-icon>shopping_cart</mat-icon><h3>Your cart is empty</h3><p>Add courses from the catalog to enroll.</p>
    <a mat-flat-button color="primary" routerLink="/customer-view-courses">Browse courses</a>
  </div>

  <div class="form-error" role="alert" id="ownedWarning" *ngIf="ownedInCart.length > 0"><mat-icon>info</mat-icon>You are already enrolled in {{ ownedInCart.length
  === 1 ? 'one course' : ownedInCart.length + ' courses' }} in your cart. Remove {{ ownedInCart.length === 1 ? 'it' : 'them' }} to continue.</div>

  <div class="cart-layout" *ngIf="courses.length > 0">
    <div class="cart-list">
      <article class="cart-item" *ngFor="let course of courses">
        <div class="cart-img"><img [src]="course.courseImageUrl" [alt]="course.courseType" (error)="$_any($event.target).style.display='none'" /></div>
        <div class="cart-info">
          <h3>{{ course.courseType }}<span class="owned-tag" *ngIf="course.courseId != null && owned.has(course.courseId)">Already enrolled</span></h3>
          <p>{{ course.courseDetails }}</p>
          <span class="price">{{ course.coursePrice | currency:'INR':'symbol':'1.0-2' }}</span>
        </div>
        <button mat-stroked-button color="warn" class="remove-btn" (click)="remove(course)"><mat-icon>delete</mat-icon>Remove from Cart</button>
      </article>
    </div>
    <aside class="summary">
      <h3>Summary</h3>
      <div class="row"><span>Courses</span><strong>{{ courses.length }}</strong></div>
      <div class="row total"><span>Total</span><strong id="cartTotal">{{ totalAmount | currency:'INR':'symbol':'1.0-2' }}</strong></div>
      <button mat-flat-button color="primary" id="enrollNow" [disabled]="ownedInCart.length > 0" (click)="enrollNow()">Enroll Now</button>
    </aside>
  </div>
</div>

```

angularapp/src/app/components/place-order/place-order.component.ts

```

import { Component, OnInit } from '@angular/core';
import { FormBuilder, FormGroup, Validators } from '@angular/forms';
import { Router } from '@angular/router';
import { MatDialog } from '@angular/material/dialog';
import { ConfirmDialogComponent } from '../confirm-dialog/confirm-dialog.component';
import { Course } from '../../models/course.model';
import { CartService } from '../../services/cart.service';
import { CustomerService } from '../../services/customer.service';
import { OrderService } from '../../services/order.service';

@Component({
  selector: 'app-place-order',
  templateUrl: './place-order.component.html',
  styleUrls: ['./place-order.component.css']
})

```

```

export class PlaceOrderComponent implements OnInit {
  orderForm: FormGroup;
  courses: Course[] = [];
  errorMessage = '';
  loading = false;
  paymentDone = false;

  constructor(
    private fb: FormBuilder,
    private cartService: CartService,
    private customerService: CustomerService,
    private orderService: OrderService,
    private router: Router,
    private dialog: MatDialog
  ) {
    this.orderForm = this.fb.group({
      name: ['', [Validators.required]],
      mobileNumber: ['', [Validators.required, Validators.pattern(/^[0-9]{10}$/)]],
      information: ['', [Validators.required]],
      email: ['', [Validators.required, Validators.email]],
      totalPrice: [{ value: 0, disabled: true }]
    });
  }

  get f() { return this.orderForm.controls; }

  ngOnInit(): void {
    this.cartService.getAllCoursesFromCart().subscribe({
      next: (cart) => {
        this.courses = cart.courses || [];
        this.orderForm.patchValue({ totalPrice: cart.totalAmount || 0 });
      },
      error: () => (this.errorMessage = 'Could not load your cart.')
    });
    this.customerService.viewCustomerById().subscribe({
      next: (customer) => this.orderForm.patchValue({
        name: customer.customerName,
        information: customer.information,
        mobileNumber: customer.user?.mobileNumber,
        email: customer.user?.email
      })
    });
  }

  makePayment(): void {
    if (this.orderForm.invalid || this.courses.length === 0 || this.loading) { return; }
    this.loading = true;
    this.errorMessage = '';
    const order = {
      customer: { customerId: Number(localStorage.getItem('customerId')) },
      courses: this.courses.map(c => ({ courseId: c.courseId })),
      orderPrice: this.orderForm.getRawValue().totalPrice
    };
    this.orderService.addOrder(order).subscribe({
      next: () => {
        // the order is saved, so empty the cart before confirming
        this.cartService.removeAllCourses().subscribe({
          next: () => { this.loading = false; this.showConfirmation(); },
          error: () => { this.loading = false; this.showConfirmation(); }
        });
      },
      error: (err) => {
        this.loading = false;
        // 409 = the server refused because a course in the cart was already bought
        this.errorMessage = err.status === 409
          ? (err.error?.message || 'You are already enrolled in one of these courses.')
          : 'Payment could not be completed. Please try again.';
      }
    });
  }

  private showConfirmation(): void {
    this.paymentDone = true;
    this.dialog.open(ConfirmDialogComponent, {
      disableClose: true,
      data: { title: 'Payment was successful!', message: 'You are now enrolled. Find your courses under My Enrollments.', confirmText: 'OK', hideCancel: true,
        icon: 'check_circle' }
    }).afterClosed().subscribe(() => this.closeConfirmation());
  }

  closeConfirmation(): void {
    this.paymentDone = false;
    this.router.navigate(['/my-orders']);
  }
}

```

F11. Delete my review

The Add Review page now has a **My reviews** box under the form. Each review has a **Delete** button that asks for confirmation first.

angularapp/src/app/components/add-review/add-review.component.ts

```

import { Component, OnInit } from '@angular/core';
import { MatDialog } from '@angular/material/dialog';
import { MatSnackBar } from '@angular/material/snack-bar';
import { FormBuilder, FormGroup, FormGroupDirective, Validators } from '@angular/forms';
import { CartService } from '../../../services/cart.service';
import { ConfirmDialogComponent } from '../../../confirm-dialog/confirm-dialog.component';
import { Review } from '../../../models/review.model';
import { OrderService } from '../../../services/order.service';
import { ReviewService } from '../../../services/review.service';

@Component({
  selector: 'app-add-review',
  templateUrl: './add-review.component.html',

```

```

styleUrls: ['./add-review.component.css']
})
export class AddReviewComponent implements OnInit {
  reviewForm: FormGroup;
  courseTypes: string[] = [];
  stars = [1, 2, 3, 4, 5];
  customerId = localStorage.getItem('customerId') || '';
  message = '';
  errorMessage = '';
  loading = false;
  myReviews: Review[] = [];

  constructor(private fb: FormBuilder, private cartService: CartService, private orderService: OrderService, private snack: MatSnackBar,
    private reviewService: ReviewService, private dialog: MatDialog) {
    this.reviewForm = this.fb.group({
      subject: ['', [Validators.required]],
      body: ['', [Validators.required]],
      rating: [0, [Validators.required, Validators.min(1), Validators.max(5)]]
    });
  }

  get f() { return this.reviewForm.controls; }

  ngOnInit(): void {
    this.loadMyReviews();
    // only courses the customer has enrolled in can be reviewed
    this.orderService.viewOrderByCustomerId().subscribe({
      next: (orders) => {
        const types = new Set<string>();
        orders.forEach((o: any) => (o.courses || []).forEach((c: any) => types.add(c.courseType)));
        this.courseTypes = Array.from(types);
      },
      error: () => (this.courseTypes = [])
    });
  }

  loadMyReviews(): void {
    this.reviewService.getMyReviews().subscribe(list => (this.myReviews = list || []));
  }

  /** A customer can remove a review they wrote. */
  deleteReview(review: Review): void {
    this.dialog.open(ConfirmDialogComponent, {
      data: { title: 'Delete your review?', message: `This removes your review of ${review.subject}.`, confirmText: 'Delete', cancelText: 'Cancel', icon: 'delete', danger: true }
    }).afterClosed().subscribe(yes => {
      if (!yes || review.reviewId == null) { return; }
      this.reviewService.deleteMyReview(review.reviewId).subscribe({
        next: () => { this.snack.open('Review deleted', 'OK', { duration: 2500 }); this.loadMyReviews(); },
        error: () => (this.errorMessage = 'Could not delete the review. Please try again.')
      });
    });
  }

  setRating(value: number): void {
    this.reviewForm.patchValue({ rating: value });
    this.reviewForm.get('rating')?.markAsTouched();
  }

  onSubmit(formDirective: FormGroupDirective): void {
    if (this.reviewForm.invalid || this.loading) { return; }
    this.loading = true;
    this.message = '';
    this.errorMessage = '';
    const review = { ...this.reviewForm.value, customer: { customerId: Number(this.customerId) } };
    this.cartService.addReview(review).subscribe({
      next: () => {
        this.loading = false;
        this.message = 'Review added successfully!';
        this.snack.open(this.message, 'OK', { duration: 3000 });
        formDirective.resetForm({ subject: '', body: '', rating: 0 });
        this.loadMyReviews();
      },
      error: () => {
        this.loading = false;
        this.errorMessage = 'Could not submit your review. Please try again.';
      }
    });
  }
}

```

angularapp/src/app/components/add-review/add-review.component.html

```

<div class="page">
  <div class="form-card">
    <h2>Add Review Details</h2>
    <p class="lead">You can review the courses you have enrolled in.</p>
    <form [formGroup]="reviewForm" #formDir="ngForm" (ngSubmit)="onSubmit(formDir)" novalidate>
      <mat-form-field appearance="outline">
        <mat-label>Customer Id</mat-label>
        <input matInput id="customerId" type="text" [value]="customerId" readonly />
      </mat-form-field>

      <mat-form-field appearance="outline">
        <mat-label>Course Type</mat-label>
        <mat-select id="subject" formControlName="subject">
          <mat-option *ngFor="let t of courseTypes" [value]="t">{{ t }}</mat-option>
        </mat-select>
        <mat-hint *ngIf="courseTypes.length === 0">Enroll in a course first to review it</mat-hint>
        <mat-error *ngIf="f['subject'].errors?.['required']">Course type is required</mat-error>
      </mat-form-field>

      <mat-form-field appearance="outline">
        <mat-label>Feedback comments</mat-label>
        <textarea matInput id="body" rows="3" formControlName="body"></textarea>
        <mat-error *ngIf="f['body'].errors?.['required']">Feedback is required</mat-error>
      </mat-form-field>
    </form>
  </div>
</div>

```

```

<div class="rating">
  <span class="rating-label" id="ratingLabel">Rating *</span>
  <div class="stars-input" id="ratingStars" role="radiogroup" aria-labelledby="ratingLabel">
    <button type="button" mat-icon-button class="star" *ngFor="let s of stars" [class.on]="s <= (f['rating'].value || 0)"
      role="radio" [attr.aria-checked]="s === f['rating'].value" [attr.aria-label]="s + ' star' + (s > 1 ? 's' : '')" (click)="setRating(s)">
      <mat-icon>{{ s <= (f['rating'].value || 0) ? 'star' : 'star_border' }}</mat-icon>
    </button>
  </div>
  <div class="form-error small" *ngIf="f['rating'].touched && f['rating'].errors">Please select a rating</div>
</div>

<div class="form-note ok" *ngIf="message">{{ message }}</div>
<div class="form-error" role="alert" *ngIf="errorMessage"><mat-icon>error_outline</mat-icon>{{ errorMessage }}</div>
<div class="form-actions">
  <button mat-flat-button color="primary" type="submit" id="submitReview" [disabled]="reviewForm.invalid || loading">Submit</button>
</div>
</form>
</div>

<div class="form-card my-reviews" id="myReviews" *ngIf="myReviews.length > 0">
  <h2>My reviews</h2>
  <p class="lead">You can remove a review you wrote.</p>
  <article class="my-review" *ngFor="let r of myReviews">
    <div class="my-review-body">
      <strong>{{ r.subject }}</strong>
      <app-star-rating [value]="r.rating || 0" [count]="1"></app-star-rating>
      <p>{{ r.body }}</p>
      <small class="muted">{{ r.dateCreated | date:'mediumDate' }}</small>
    </div>
    <button mat-stroked-button color="warn" type="button" class="del-review" (click)="deleteReview(r)"><mat-icon>delete</mat-icon>Delete</button>
  </article>
</div>
</div>

```

F12. CSS to append (Midnight look)

Add each block at the **bottom** of the file named above it. Do not replace the files.

Append to angularapp/src/styles.css

```

/* image upload on the add / edit course forms (the preview above reuses .img-preview) */
.upload { margin-bottom: 14px; }
.upload .img-preview { margin-bottom: 12px; }
.upload-row { display: flex; flex-wrap: wrap; align-items: center; gap: 12px; }
.upload-hint { color: var(--muted); font-size: 13px; }

```

Append to angularapp/src/app/components/registration/registration.component.css

```

/* live password checklist */
.pw-rules { display: grid; gap: 6px; margin: 2px 0 14px; padding: 12px 14px; list-style: none; border-radius: 10px; background: var(--surface-2); border: 1px solid var(--line); }
.pw-rules li { display: flex; align-items: center; gap: 8px; color: var(--muted); font-size: 13.5px; }
.pw-rules li mat-icon { width: 18px; height: 18px; font-size: 18px; }
.pw-rules li.ok { color: var(--ok); }

```

Append to angularapp/src/app/shared/course-card/course-card.component.css

```

/* "Enrolled" badge on the picture and the Go to course button */
.enrolled-badge { position: absolute; top: 10px; left: 10px; z-index: 2; display: inline-flex; align-items: center; gap: 4px; padding: 4px 10px 4px 6px; border-radius: 999px; background: rgba(11, 18, 32, .82); border: 1px solid rgba(52, 211, 153, .5); color: var(--ok); font: 700 12.5px var(--display); backdrop-filter: blur(6px); }
.enrolled-badge mat-icon { width: 18px; height: 18px; font-size: 18px; }
.go-course { width: 100%; }

```

Append to angularapp/src/app/components/course-details/course-details.component.css

```

/* already enrolled: status box instead of the buy buttons, and the delete button on my own review */
.enrolled-box { display: grid; gap: 12px; }
.enrolled-status { display: flex; align-items: center; gap: 12px; padding: 12px 14px; border-radius: 12px; background: rgba(52, 211, 153, .1); border: 1px solid rgba(52, 211, 153, .35); }
.enrolled-status mat-icon { flex: none; color: var(--ok); }
.enrolled-status strong { display: block; color: var(--ink); }
.enrolled-status span { color: var(--muted); font-size: 13.5px; }
.del-review { flex: none; margin-left: auto; align-self: flex-start; }

```

Append to angularapp/src/app/components/add-review/add-review.component.css

```

/* my reviews list (delete one of my own reviews) */
.my-reviews { margin-top: 24px; }
.my-reviews h2 { font-size: 22px; }
.my-review { display: flex; align-items: flex-start; gap: 12px; padding: 14px 0; border-top: 1px solid var(--line); }
.my-review:first-of-type { border-top: 0; }
.my-review-body { flex: 1; min-width: 0; }
.my-review-body p { margin: 6px 0 4px; color: var(--ink-2); overflow-wrap: anywhere; }
.my-review .del-review { flex: none; }

```

Append to angularapp/src/app/components/my-cart/my-cart.component.css

```
/* "Already enrolled" tag next to a course title in the cart */
.owned-tag { display: inline-block; margin-left: 10px; padding: 2px 10px; border-radius: 999px; background: rgba(248, 113, 113, .12); border: 1px solid rgba(248, 113, 113, .35); color: var(--danger); font: 600 12px var(--body, inherit); vertical-align: middle; }
```

What I tested (browser, mock backend, Midnight look)

Registration: valid data enables the button; a username with a space, a@b, a@b., a b@x.com, a.b@c.d, a mobile starting with 5, a 9-digit mobile, and passwords with no upper, no lower, no digit, no special character, a space, or too short are all blocked; mobiles starting 6, 7, 8 and 9 are accepted; the checklist shows ticks. Images: the link box is gone; a text file is rejected with a message; a real image uploads, shows a preview, enables the button, and the saved course holds the server URL; the edit page shows the current image with **Change image**. Purchase: before buying there is no "Go to course"; after paying, that course shows Enrolled and Go to course and has no Add to Cart, other courses still show Add to Cart; the course page shows "You are enrolled"; the server answers 409 to a second purchase; a bought course sitting in the cart shows the warning and disables Enroll Now. Reviews: My reviews appears after adding one; the delete icon shows on the course page; deleting asks for confirmation, removes it and shows "Review deleted". Footer: shows on every page with the right links, no sideways scrolling on a phone.

Run it

```
cd springapp && mvn spring-boot:run
cd angularapp && npm start
```

Log in as admin, open Add Course, click Browse image, pick a picture, add the course, and check the picture on the course card. If the picture does not show, open the picture link from the course in a new tab to see whether the backend serves `/uploads/...`